Table of Contents

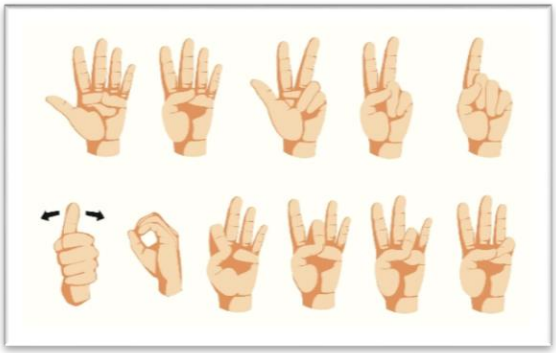
CHAPTER 1
PRELIMINARY INVESTIGATION

1.1 Introduction to Sign Language Communication

Overview

GestuX Gloves is developed as an assistive communication system aimed at minimizing the communication gap between deaf and mute individuals and the general public. Effective communication plays a vital role in education, healthcare, employment, and social interaction. However, individuals who rely on sign language often face difficulty when interacting with people unfamiliar with it.

Sign language is a structured and complete form of communication that uses hand gestures, facial expressions, and body movements to convey meaning. Although it is widely practiced within the deaf community, it is not universally understood, which creates communication barriers in everyday interactions.



The above representation shows how alphabets and numbers are expressed through specific hand gestures. While effective among trained individuals, communication becomes difficult when interacting with people unfamiliar with these gestures.

Communication Vision

The vision of **GestuX Gloves** is to provide a portable and affordable **gesture-to-voice conversion system** that operates in real time. Instead of relying on complex camera-based recognition or expensive flex sensors, this project adopts a simplified embedded hardware approach.

By integrating an **Arduino-based controller**, motion sensing technology, and wireless transmission, the system converts physical gestures into audible speech through a mobile device. This enhances accessibility and supports independent communication.

Core Communication Principles

- **Gesture-Based Recognition**
  The system uses predefined gesture combinations formed through finger contact detection and motion sensing. Each gesture corresponds to a specific message, ensuring structured and reliable communication.
- **Cost-Effective Hardware Design**
  A combinational contact-based circuit is implemented instead of high-cost flex sensors. This significantly reduces the overall cost of the device while maintaining functional efficiency.
- **Real-Time Processing and Output**
  The Arduino continuously monitors sensor inputs and processes gestures instantly. The detected message is displayed on the LCD and transmitted wirelessly without noticeable delay.
- **Wireless Voice Conversion**
  Through the HC-05 Bluetooth module, processed data is sent to a mobile application. The application converts the received text into speech using Text-to-Speech technology.

1.2 About the Project – GestuX Gloves

Project Overview

GestuX Gloves is a wearable assistive communication device designed to convert hand gestures into text and voice output. The system focuses on enabling deaf and mute individuals to communicate effectively with people who do not understand sign language. It combines embedded hardware components with wireless communication to deliver real-time gesture-to-speech conversion.

The project is built around a simplified hardware architecture that avoids the use of costly flex sensors. Instead, it uses a combinational contact-based detection mechanism along with motion sensing to recognize predefined gestures.

System Working Concept

The working principle of GestuX Gloves is based on detecting finger movements and hand orientation. When a user performs a predefined gesture, electrical contact is established between specific metal points placed at finger joints. These signals are sent to an Arduino Uno, which processes the input and determines the corresponding message.

Once processed, the output is displayed on a 16×2 LCD display and simultaneously transmitted via the HC-05 Bluetooth module to a mobile device. The mobile application then converts the received text into speech using Text-to-Speech functionality.

Key Components of the Project

- **Arduino Uno**
  Acts as the central processing unit of the system. It reads input signals from the contact circuit and motion sensor, processes gesture combinations, and generates the corresponding output.

- **Combinational Contact Circuit**
  Metal contact points are placed at finger joints to detect gesture combinations. This replaces expensive flex sensors and significantly reduces overall system cost.
- **MPU6050 Motion Sensor**
  The accelerometer and gyroscope module detects hand orientation and movement. It improves accuracy by identifying motion-based gestures.
- **HC-05 Bluetooth Module**
  Enables wireless communication between the glove system and the mobile application, allowing seamless transmission of processed data.
- **16×2 LCD Display**
  Displays the detected gesture message instantly, providing visual confirmation to the user.

Project Objective

The primary objective of GestuX Gloves is to design a cost-effective, portable, and reliable communication system that enhances accessibility for deaf and mute individuals. The system aims to promote independence, reduce dependency on interpreters, and provide real-time communication support in everyday environments.

1.3 Need for the System

Communication Gap in Society

In today's interconnected world, communication is essential for participation in education, employment, healthcare, and daily social activities. However, deaf and mute individuals often face challenges when interacting with people who do not understand sign language. This creates a significant communication gap that limits independence and confidence.

Although sign language is an effective medium within the deaf community, it is not widely known among the general population. As a result, individuals frequently depend on interpreters or written communication, which may not always be practical in real-time situations

Limitations of Existing Communication Methods

Traditional communication methods such as writing on paper or using mobile text messages can be slow and inconvenient during face-to-face interaction. In urgent situations, delays in communication may cause misunderstandings or critical issues.

Existing technological solutions often rely on expensive **flex sensors**, complex image processing systems, or machine learning algorithms. These systems increase overall cost and require advanced hardware or software infrastructure, making them less accessible to a large section of society.

Requirement for a Practical Solution

There is a clear need for a simple, wearable, and affordable system that can convert gestures into voice output instantly. Such a device should operate in real time and should not depend on heavy computational systems or high-cost components.

A practical solution must meet the following requirements:

- **Affordability**
  The system should be economically feasible so that it can be adopted by individuals from different financial backgrounds.
- **Portability**
  The device should be lightweight and wearable, allowing users to carry and operate it comfortably throughout the day.
- **Real-Time Communication**
  Gesture detection and speech output must occur without noticeable delay to ensure smooth interaction.
- **Ease of Use**
  The system should be simple to operate and should not require advanced technical knowledge from the user.

Conclusion
The growing need for inclusive communication tools highlights the importance of developing an affordable and efficient gesture-to-voice conversion system. GestuX Gloves addresses this need by combining embedded hardware, motion sensing, and wireless communication to create a practical solution for everyday interaction.

1.4 Limitations of Existing Systems

Overview of Existing Solutions

Several technological solutions have been developed to convert sign language into text or speech. These systems aim to assist deaf and mute individuals by recognizing hand gestures using various sensing techniques. While the intention behind these systems is positive, many of them face practical limitations in terms of cost, complexity, and usability.

Most existing systems depend on advanced hardware components such as flex sensors, camera-based recognition systems, or machine learning algorithms. Although these technologies can provide accurate gesture detection, they often increase the overall complexity of the device.

Technical Limitations

Many gesture recognition systems rely on **flex sensors** that detect finger bending through resistance changes. These sensors are expensive and may wear out over time due to continuous usage. Additionally, calibration of flex sensors can be complicated and may require precise adjustments.

Camera-based systems use image processing and pattern recognition techniques to identify gestures. However, they require high computational power, controlled lighting conditions, and powerful processors, making them less practical for portable everyday use.

Financial Constraints

One of the major drawbacks of existing systems is their high cost. Devices that use multiple flex sensors or advanced image processing modules can become financially inaccessible for many users. This contradicts the primary objective of assistive communication technology, which is to provide support to a wider section of society.

High manufacturing and maintenance costs also limit large-scale implementation, especially in developing regions where affordability is a key concern.

Operational Challenges

Some systems are bulky and not comfortable for daily wear. Complex wiring, heavy hardware, and battery consumption reduce portability and convenience. In addition, certain solutions require technical expertise for setup and operation, making them less user-friendly.

Reliability issues such as signal noise, inaccurate gesture mapping, and delay in processing can further affect performance and user confidence.

Conclusion

Although existing sign-to-speech conversion systems demonstrate technological advancement, they are often limited by high cost, technical complexity, and operational challenges. These limitations highlight the necessity for a simplified, affordable, and efficient alternative system, which forms the foundation for the proposed GestuX Gloves solution.

1.5 Proposed System

System Concept and Design Approach

The proposed system, **GestuX Gloves**, is developed as a wearable communication device that converts predefined hand gestures into readable text and audible speech. The system is designed to overcome the technical and financial limitations observed in existing gesture recognition solutions. Instead of depending on costly flex sensors or high-computation image processing systems, the project adopts a simplified embedded hardware approach.

The design philosophy focuses on affordability, portability, and real-time response. By combining contact-based gesture detection with motion sensing and wireless transmission, the system ensures structured and reliable communication without increasing hardware complexity.

The entire system operates around the **Arduino Uno**, which serves as the central control unit. It continuously monitors input signals, processes gesture combinations, and generates corresponding output messages.

Functional Structure of the System

The operation of GestuX Gloves can be understood through three primary functional stages:

• **Gesture Input Mechanism**
Hand gestures are detected using a combinational contact circuit along with motion sensing. This stage captures both finger combinations and hand orientation.

• **Signal Processing Unit**
The Arduino processes digital inputs from the contact points and motion sensor. It compares the detected pattern with predefined logic conditions stored in the program.

• **Communication Output Unit**
Once a gesture is identified, the system displays the message on an LCD and transmits the same data wirelessly to a mobile device for speech generation.

This structured workflow ensures smooth and efficient system operation.

1.5.1 Combinational Contact-Based Circuit

Hardware-Based Gesture Detection

The combinational contact-based circuit is the primary method used for detecting finger gestures. Metal contact points are placed strategically at finger joints so that when a user performs a specific gesture, certain contact points connect and complete an electrical circuit.

Each finger is mapped to a dedicated digital input pin of the Arduino. When contact is established, the corresponding input pin registers a signal change. By analyzing combinations of these signals, the Arduino identifies predefined gesture patterns.

Unlike flex sensors, which measure bending resistance and require calibration, the contact-based method generates direct digital signals. This improves reliability and simplifies implementation.

Advantages of Contact-Based Design

• **Reduced Cost**
Metal contact points and basic wiring are significantly cheaper than multiple flex sensors, making the system economically feasible.

• **Simplified Calibration**
Since the system works on digital contact detection, it does not require complex resistance calibration procedures.

• **Durability and Stability**
The absence of delicate flex components improves durability and reduces maintenance concerns.

• **Structured Gesture Mapping**
Predefined combinations allow clear mapping between finger patterns and output messages.

This approach ensures that gesture detection remains both affordable and practically implementable.

1.5.2 Motion Detection using MPU6050

Integration of Orientation Sensing

To enhance gesture recognition capability, the system integrates the **MPU6050 motion sensor**, which contains a three-axis accelerometer and a three-axis gyroscope. This module detects hand orientation, tilt, and angular movement.

The sensor communicates with the Arduino using the I2C communication protocol. It continuously provides acceleration and rotational data, which the Arduino interprets to determine hand position and movement direction.

Certain gestures depend not only on finger contact but also on hand orientation. For example, tilting the hand forward or sideways within defined angular thresholds can trigger additional predefined messages.

Benefits of Motion Integration

• **Improved Accuracy**
Combining motion sensing with contact detection reduces false gesture recognition.

• **Expanded Gesture Range**
Orientation-based gestures increase the total number of possible commands.

• **Real-Time Data Monitoring**
The sensor continuously updates movement data, ensuring immediate detection.

• **Better Reliability**
Dual-sensing mechanisms provide more stable and dependable output.

The inclusion of the MPU6050 enhances the system's functionality without significantly increasing complexity.

1.5.3 LCD and Bluetooth Communication

Visual and Wireless Output Mechanism

After gesture detection and processing, the output stage is activated. The detected message is first displayed on a **16×2 LCD display** connected to the Arduino. This provides instant visual feedback, allowing the user to confirm the detected gesture.

Simultaneously, the Arduino transmits the processed text through the **HC-05 Bluetooth module** using serial communication. The Bluetooth module establishes a wireless link between the glove system and a mobile device.

On the mobile device, an application developed using **MIT App Inventor** receives the transmitted text. The application then uses built-in Text-to-Speech functionality to convert the message into audible speech.

Dual Output Advantages

• **Immediate Visual Confirmation**
The LCD ensures that the user can verify the detected message instantly.

• **Wireless Data Transmission**
Bluetooth communication enables cable-free connectivity and mobility.

• **Audible Communication Support**
Text-to-Speech conversion ensures that people unfamiliar with sign language can understand the message clearly.

• **Enhanced Practicality**
The combination of visual and voice output makes the system suitable for real-world interaction scenarios.

Final Remarks on the Proposed System

The proposed GestuX Gloves system combines contact-based detection, motion sensing, and wireless communication into a unified embedded solution. By focusing on simplicity, affordability, and real-time processing, the system provides an effective alternative to expensive and complex gesture recognition technologies.

This design ensures that communication becomes more accessible, portable, and practical for everyday use.

1.6 Scope of the Project

Purpose and Applicability

The scope of **GestuX Gloves** focuses on developing a practical and affordable assistive communication system for deaf and mute individuals. The project is designed to bridge the communication gap between sign language users and people who are not familiar with sign language. By converting predefined gestures into text and speech output, the system enables smoother interaction in daily life.

The device is intended for real-time usage in environments such as educational institutions, workplaces, healthcare facilities, and public spaces. Its wearable nature ensures that users can operate it comfortably throughout the day.

Functional Boundaries

The system operates within a predefined set of gesture combinations programmed into the Arduino microcontroller. Each gesture corresponds to a specific phrase or command stored in the program. The device does not perform complex language translation but focuses on structured and reliable gesture-to-message conversion.

The scope includes:

- **Predefined Gesture Mapping**
  The system supports single, double, and multiple finger combinations, along with motion-based gestures detected through the MPU6050 sensor.
- **Real-Time Processing and Output**
  The device detects gestures instantly and provides both visual output on the LCD and audible output through a mobile application.
- **Wireless Communication Support**
  Bluetooth integration allows seamless connection between the glove system and an Android device for speech generation.

Technical Coverage

The project covers the integration of embedded systems, sensor interfacing, serial communication, and mobile application interaction. It demonstrates how hardware and software components can work together to create a functional IoT-based assistive device.

The system also highlights:

- Microcontroller programming using Arduino IDE
- I2C communication with motion sensors
- Serial data transmission via Bluetooth
- Mobile-based Text-to-Speech implementation

Practical Limitations of Scope

While the system provides real-time gesture-to-voice conversion, it is limited to predefined gestures stored in the program. It does not include artificial intelligence or automatic sign language learning capabilities. Additionally, the accuracy of gesture detection depends on proper finger contact and defined motion thresholds.

Despite these boundaries, the project effectively demonstrates a simplified and affordable approach to assistive communication technology.

Closing Note

The scope of GestuX Gloves lies in delivering a structured, cost-effective, and wearable communication solution using embedded hardware and wireless connectivity. It serves as a foundation for future advancements in gesture recognition systems while maintaining practical implementation feasibility.

1.7 Stakeholders

Involved and Affected Parties

The development and implementation of **GestuX Gloves** involve multiple stakeholders who directly or indirectly benefit from the system.

The primary stakeholder of this project is the **developer**, who is responsible for system design, hardware integration, programming, and testing. The developer plays a central role in conceptualizing and implementing the gesture-to-voice conversion mechanism.

In addition to the developer, the intended beneficiaries of the system include **deaf and mute individuals**, who are the primary users of the device. The system is specifically designed to support their communication needs in real-time environments.

Other indirect stakeholders include academic supervisors and educational institutions, as the project contributes to research and development in embedded systems and assistive communication technology.

1.8 Technologies Used

Technology Stack Overview

The development of **GestuX Gloves** involves the integration of embedded hardware components and supporting software platforms. The system combines microcontroller programming, motion sensing, wireless communication, and mobile-based speech synthesis to achieve real-time gesture-to-voice conversion.

The technologies used in this project are categorized into **Software Technologies** and **Hardware Components**, each playing a crucial role in system implementation.

1.8.1 Software Technologies

The software tools used in the development of **GestuX Gloves** play a crucial role in programming, data processing, and mobile communication. These technologies ensure smooth interaction between hardware components and the end-user interface.

Arduino IDE

The **Arduino IDE** is used to write and upload the program to the Arduino Uno. It controls gesture detection logic, motion sensor reading, LCD display, and Bluetooth communication. It enables the microcontroller to process input signals and generate output.

MIT App Inventor

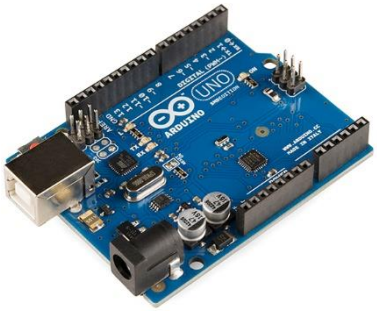
MIT App Inventor is used to develop the Android application for the project. It receives Bluetooth data from the glove and converts the text into speech using Text-to-Speech functionality. It enables wireless voice output for gesture communication.

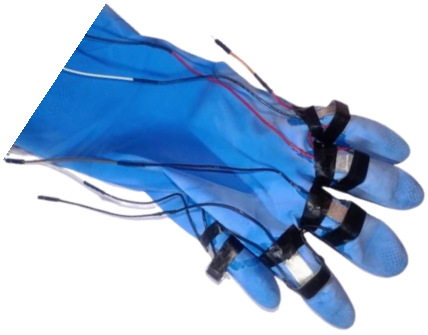
1.8.2 Hardware Components

The hardware components used in **GestuX Gloves** form the physical foundation of the system. These components work together to detect hand gestures, process input signals, and generate both visual and voice outputs. The selection of hardware was based on affordability, reliability, and ease of integration. Each component plays a specific role in ensuring smooth and real-time gesture-to-voice communication.

Arduino Uno

The **Arduino Uno** acts as the main controller of the system. It processes input signals from sensors and controls the LCD and Bluetooth module. It executes the programmed gesture recognition logic.





Combinational Contact Circuit

The contact-based circuit detects finger combinations through metal contact points. When contacts connect, digital signals are sent to the Arduino. It replaces costly flex sensors and reduces system cost.

MPU6050 Motion Sensor

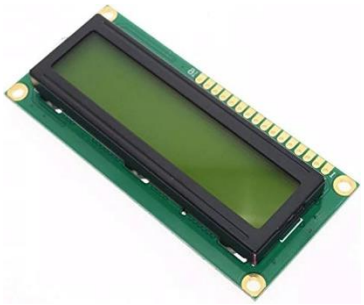
The **MPU6050** detects hand orientation and movement using accelerometer and gyroscope data. It improves gesture accuracy by monitoring tilt and direction. It communicates with Arduino using I2C protocol.

HC-05 Bluetooth Module

The **HC-05** enables wireless communication between the glove and mobile device. It transmits processed gesture data via serial communication. This allows real-time speech output on the phone.

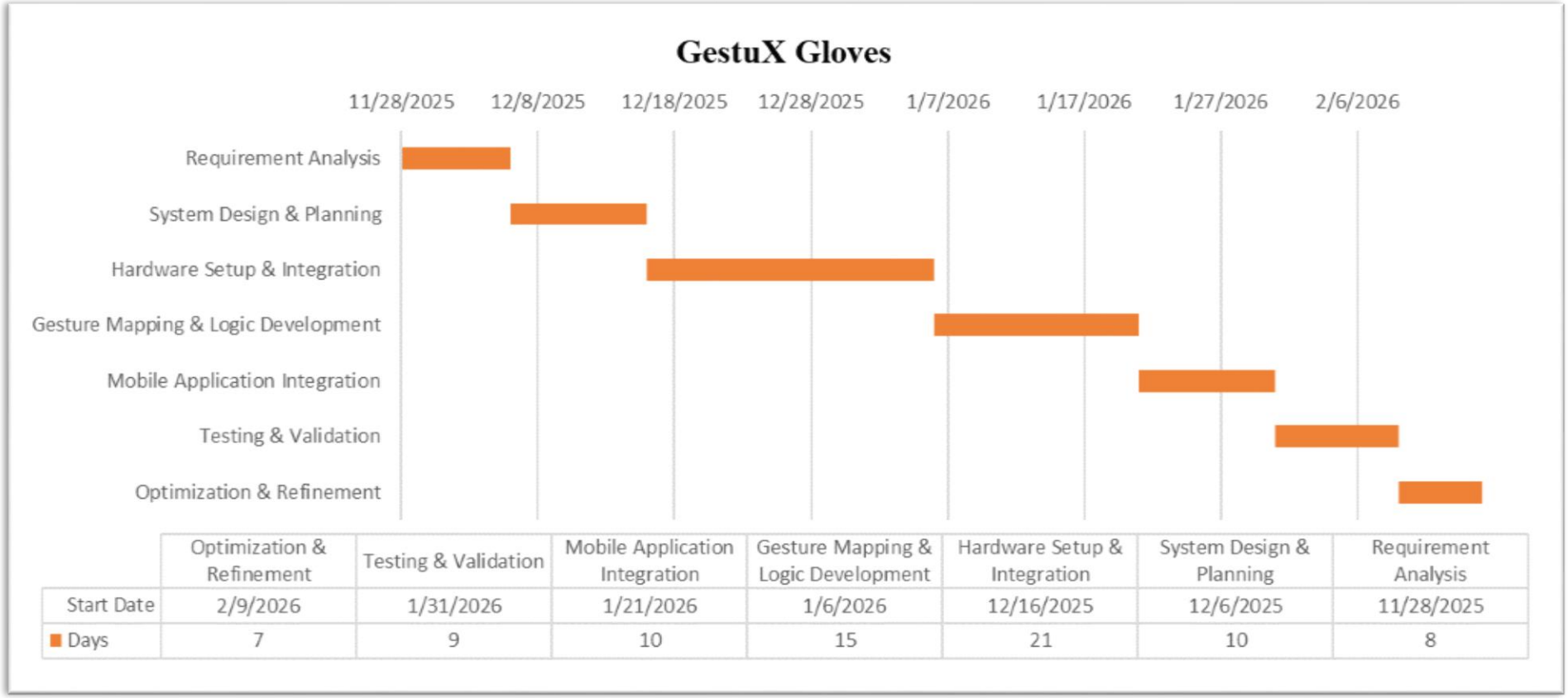
16×2 LCD Display

The **16×2 LCD** displays the detected message instantly. It provides visual confirmation of the performed gesture. It enhances usability and system feedback.



1.9 Gantt Chart

Task Name	Start Date	End Date	Days	Duration (weeks)
Requirement Analysis	11/28/2025	12/5/2025	8	1 week
System Design & Planning	12/6/2025	12/15/2025	10	1.5 weeks
Hardware Setup & Integration	12/16/2025	1/5/2026	21	3 weeks
Gesture Mapping & Logic Development	1/6/2026	1/20/2026	15	2 weeks
Mobile Application Integration	1/21/2026	1/30/2026	10	1.5 weeks
Testing & Validation	1/31/2026	2/8/2026	9	1.5 weeks
Optimization & Refinement	2/9/2026	2/15/2026	7	1 week



	Optimization & Refinement	Testing & Validation	Mobile Application Integration	Gesture Mapping & Logic Development	Hardware Setup & Integration	System Design & Planning	Requirement Analysis
Start Date	2/9/2026	1/31/2026	1/21/2026	1/6/2026	12/16/2025	12/6/2025	11/28/2025
Days	7	9	10	15	21	10	8

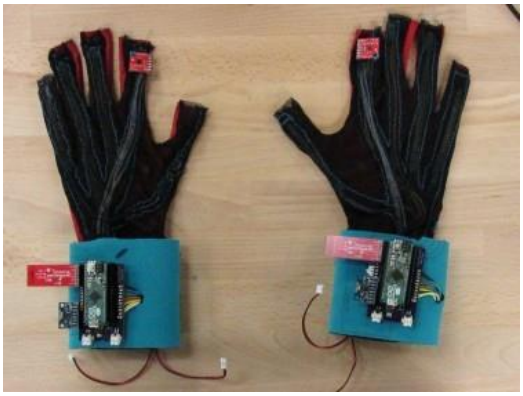
CHAPTER 2
LITERATURE SURVEY

2.1 Lemelson-MIT Sign Language Converter (SignAloud)

One of the most recognized wearable sign language translation systems was developed by **Navid Azodi** and **Thomas Pryor**, students from the University of Washington. Their project, called **SignAloud**, won the Lemelson-MIT Student Prize in 2016. The invention was designed to translate American Sign Language (ASL) gestures into spoken words in real time.

The system consisted of sensor-based gloves that detected hand position and finger movements. These sensors transmitted data wirelessly to a central processing unit. The collected gesture data was analyzed using statistical algorithms, and when a gesture matched a stored pattern, the corresponding word was spoken through a speaker.

The primary objective of this invention was to eliminate the communication barrier between deaf individuals and people who do not understand sign language. It demonstrated that wearable technology can effectively convert gestures into speech.



Observations from the Study

The SignAloud system proved that real-time gesture translation is technically achievable. However, it relied on multiple advanced sensors and complex data processing methods. This increased the overall cost and complexity of the device.

The study of this system provided inspiration for developing a simplified and more affordable solution in GestuX Gloves by replacing expensive sensing mechanisms with a contact-based circuit design.

2.2 Sign Language to Speech Conversion Using Arduino

Several researchers have developed sign language translation systems using the **Arduino microcontroller** as the core processing unit. These systems typically use flex sensors, accelerometers, and contact sensors to detect finger bending and hand movements. The Arduino reads sensor inputs, processes the gesture logic, and generates corresponding text output.

In most implementations, flex sensors are attached to each finger to measure bending resistance. The analog values obtained from these sensors are compared with predefined threshold values stored in the program. Once a gesture pattern matches, the corresponding alphabet or word is displayed on an LCD screen and transmitted to a mobile device via Bluetooth for speech conversion.

The objective of these systems is to provide a portable and low-cost communication solution for deaf and mute individuals. Arduino-based systems are simpler compared to AI-based models, but they still depend heavily on sensor calibration and accurate threshold settings.

Analysis of Existing Approach

Although these systems are practical and widely implemented in academic projects, they have certain drawbacks. Flex sensors are relatively expensive and may degrade over time. Additionally, small variations in finger bending can produce inconsistent readings, requiring frequent calibration.

The study of Arduino-based gesture systems helped in understanding sensor interfacing and embedded programming. It also highlighted the importance of developing a more affordable and stable detection mechanism, which influenced the design of GestuX Gloves.

2.3 Limitations, Problems and Scope

Limitations Observed in Existing Systems

Based on the study of the Lemelson-MIT SignAloud system and various Arduino-based gesture translation projects, several limitations were identified. Many systems rely heavily on **flex sensors**, which are relatively expensive and prone to wear and tear over time. Continuous bending can reduce sensor accuracy, leading to inconsistent gesture detection.

Camera-based systems require controlled lighting conditions and higher processing power. Such systems may not perform efficiently in real-world environments and increase overall system complexity.

Another major limitation is the dependency on precise calibration. Small variations in finger movement or hand orientation can result in incorrect output if threshold values are not properly adjusted.

Problems Identified

Most existing gesture translation systems focus primarily on accuracy but overlook affordability. High component costs make these systems less accessible to a large portion of society, especially in developing regions.

Additionally, some systems are bulky and uncomfortable for daily use. Complex wiring and multiple sensors increase hardware weight and reduce portability. Maintenance and replacement costs further limit long-term usability.

Another issue is limited scalability. Many systems only recognize alphabets rather than complete phrases, requiring multiple gestures to form a sentence, which slows down communication.

Scope for Improvement

The limitations identified in existing systems highlight the need for a simplified, cost-effective, and reliable alternative. There is scope to reduce hardware dependency on expensive sensors and replace them with simpler digital detection mechanisms.

By combining a contact-based circuit with motion sensing, gesture detection can be achieved with lower cost and improved durability. Integration with mobile-based Text-to-Speech technology further enhances practicality.

The study of existing systems provided a strong foundation for developing GestuX Gloves, focusing on affordability, portability, and real-time communication efficiency.

2.4 Cost Analysis of System Components

The cost analysis of GestuX Gloves was conducted to evaluate the overall hardware expenditure involved in the development of the system. Since the primary objective of this project is to create a cost-effective assistive communication device, careful consideration was given to the selection of components. Each hardware module was chosen based on affordability, availability in the local market, and compatibility with the Arduino-based architecture.

The analysis also helps in understanding how the proposed system compares financially with existing gesture translation devices that rely on expensive flex sensors or advanced processing units. By selecting commonly available components, the overall implementation cost has been kept within a practical and manageable range.

The following table presents the list of hardware components used in the project along with their approximate cost in Indian Rupees

SR.NO	Component List	PCS	Cost in ₹ (Rupees)
1	Arduino UNO	1	₹ 600
2	MPU6050 accelerometer	1	₹ 200
3	HC-05 Bluetooth Module	1	₹ 260
4	16*2 LCD display	1	₹ 140
5	Rubber Glove	1	₹ 80
6	Male to female, Male to Male, Female to Female Connecting wire	50	₹ 150
7	Soldering Flux	1	₹ 50
8	Glue stick	15	₹ 150
9	1 k resistor	15	₹ 15
10	Connecting Wire	5m	₹ 30
11	batteries	2	₹ 300
12	1 cell & 2 Cell Holder	1	₹ 60
13	TP4056 Charging Module	1	₹ 30
14	Metal Contact plates	5	₹ 30

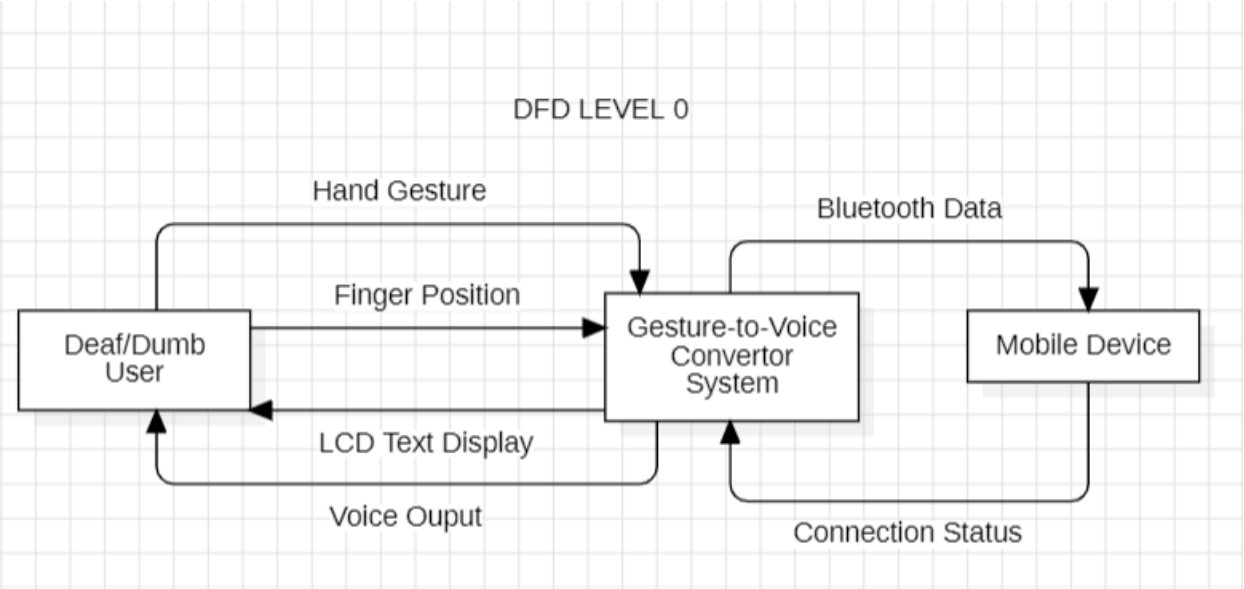
The estimated total cost demonstrates that the system is economically feasible compared to existing gesture translation systems that rely on expensive sensors. By replacing high-cost flex sensors with a combinational contact-based circuit, the overall hardware expenditure has been significantly reduced. The use of commonly available components such as Arduino Uno, MPU6050, and HC-05 Bluetooth module further contributes to maintaining a reasonable budget.

This cost-effective approach makes the project suitable for academic implementation as well as practical deployment in real-world scenarios. It also highlights that assistive communication technology can be developed without requiring high-end or industrial-grade components.

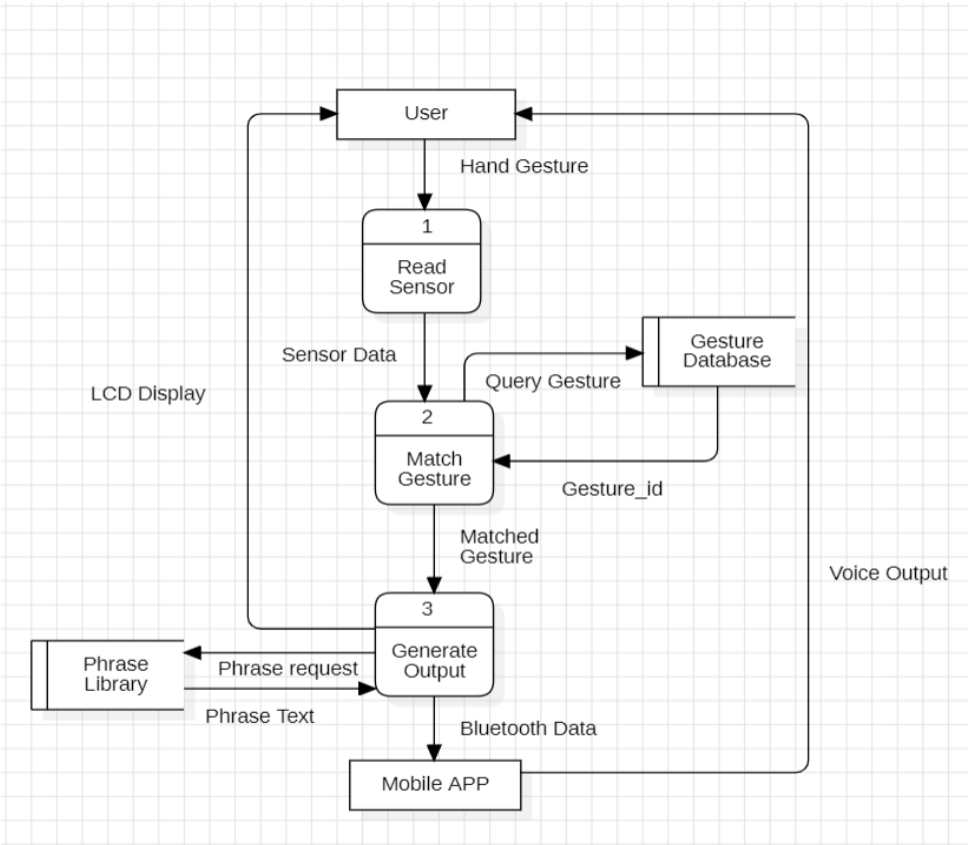
CHAPTER 3
SYSTEM DESIGN DIAGRAMS

3.1 Data Flow Diagram (DFD)

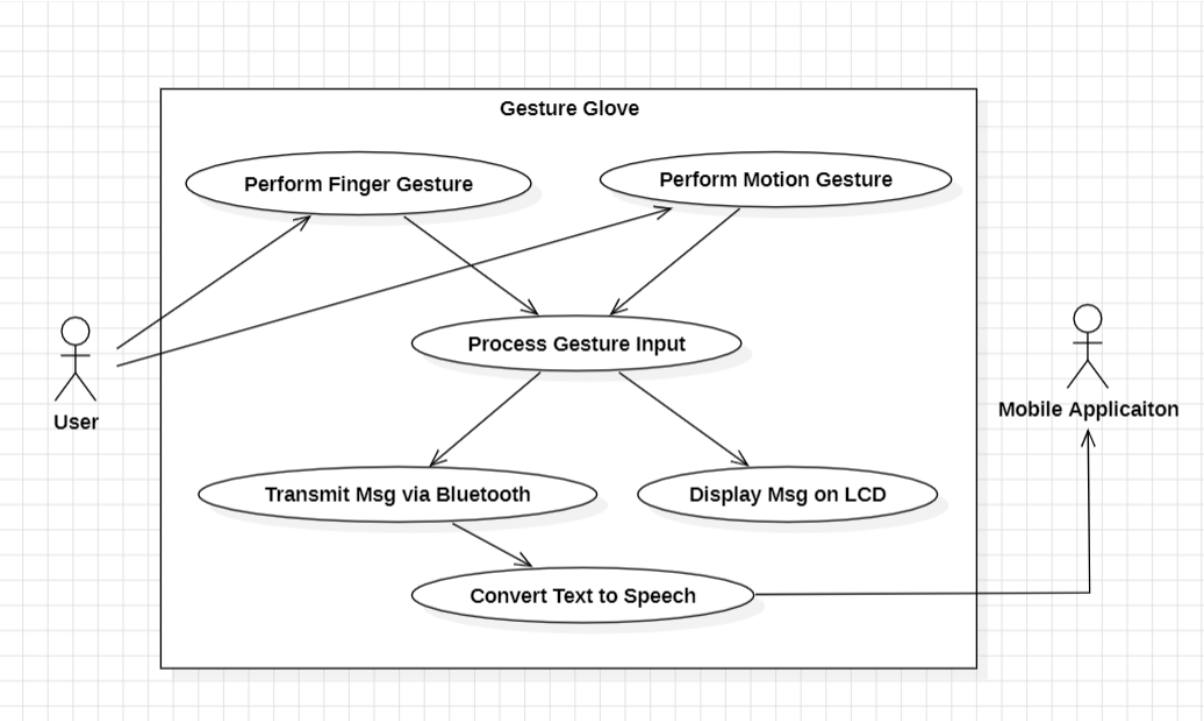
Data Flow (LEVEL 0)



Data Flow (LEVEL 1)



3.2 Use Case Diagram



3.2.1 Use Case Diagram Description

Overview

The Use Case Diagram illustrates the functional interactions between the user, the Gesture Glove system, and the Mobile Application. It represents how gestures performed by the user are captured, processed, and converted into text and speech output. The diagram highlights the major system functionalities and defines how external actors interact with the system.

Actors

- **User (Deaf/Dumb Person):** Primary actor who performs hand gestures using the glove to communicate.
- **Mobile Application:** Secondary actor that receives Bluetooth data and converts text into speech output.

Use Cases

Gesture Input

- **Perform Finger Gesture** – The user creates predefined finger combinations using the contact-based circuit embedded in the glove.
- **Perform Motion Gesture** – The user moves or tilts the hand, and the MPU6050 sensor captures orientation data (X and Y axis values).

- **Process Gesture Input** – The system analyzes the received digital and motion inputs and matches them with stored gesture patterns.

Output Generation

- **Display Message on LCD** – Once a gesture is recognized, the corresponding phrase is displayed on the 16×2 LCD screen.
- **Transmit Message via Bluetooth** – The processed phrase is sent wirelessly through the HC-05 Bluetooth module.
- **Convert Text to Speech** – The mobile application receives the transmitted text and generates audible speech using Text-to-Speech functionality.

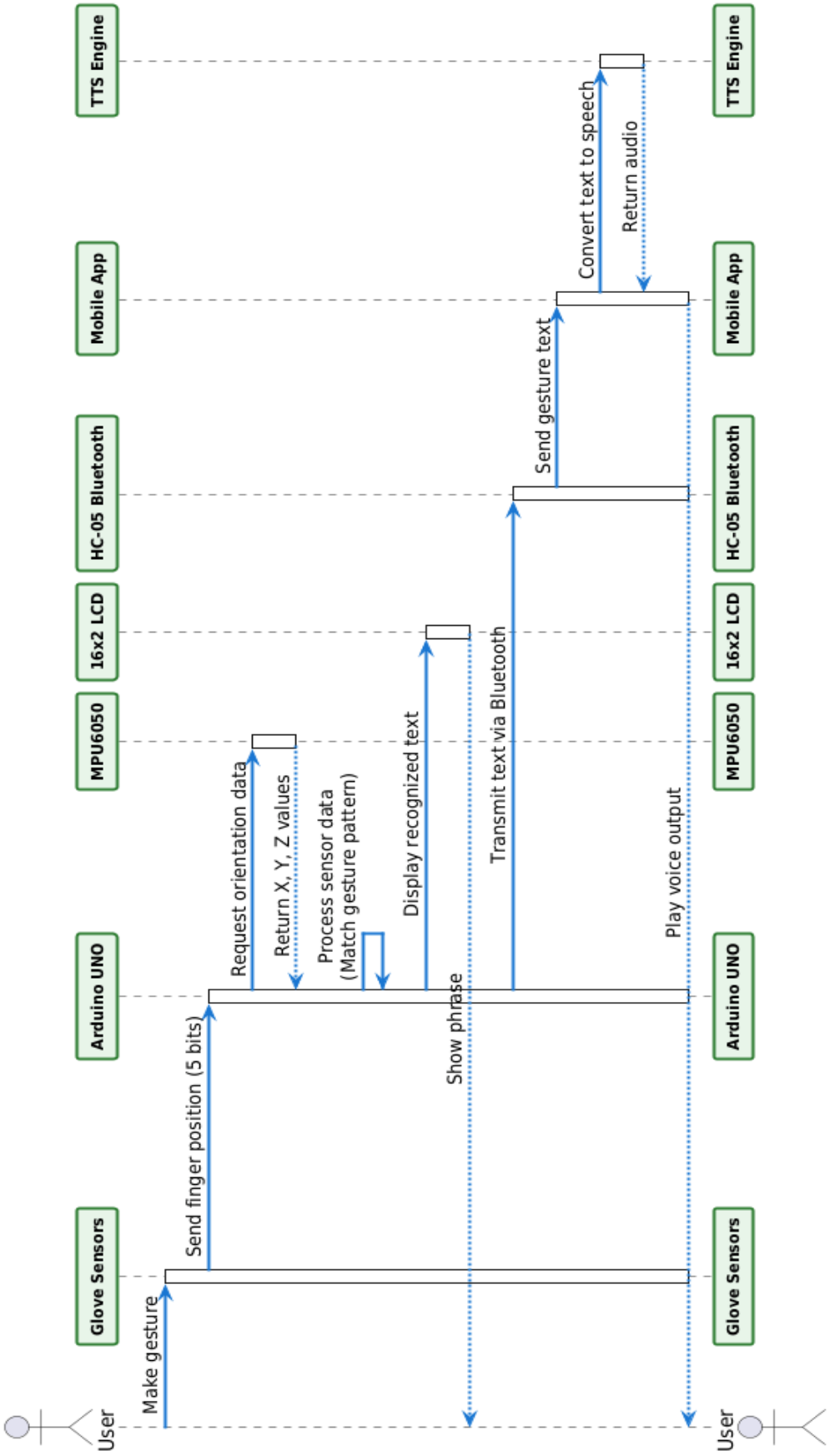
Relationships

- **Include:** Process Gesture Input includes both Perform Finger Gesture and Perform Motion Gesture as input mechanisms.
- **Include:** Transmit Message via Bluetooth is triggered after successful gesture processing.
- **Extend:** Convert Text to Speech extends Transmit Message via Bluetooth, as speech output depends on Bluetooth data reception.

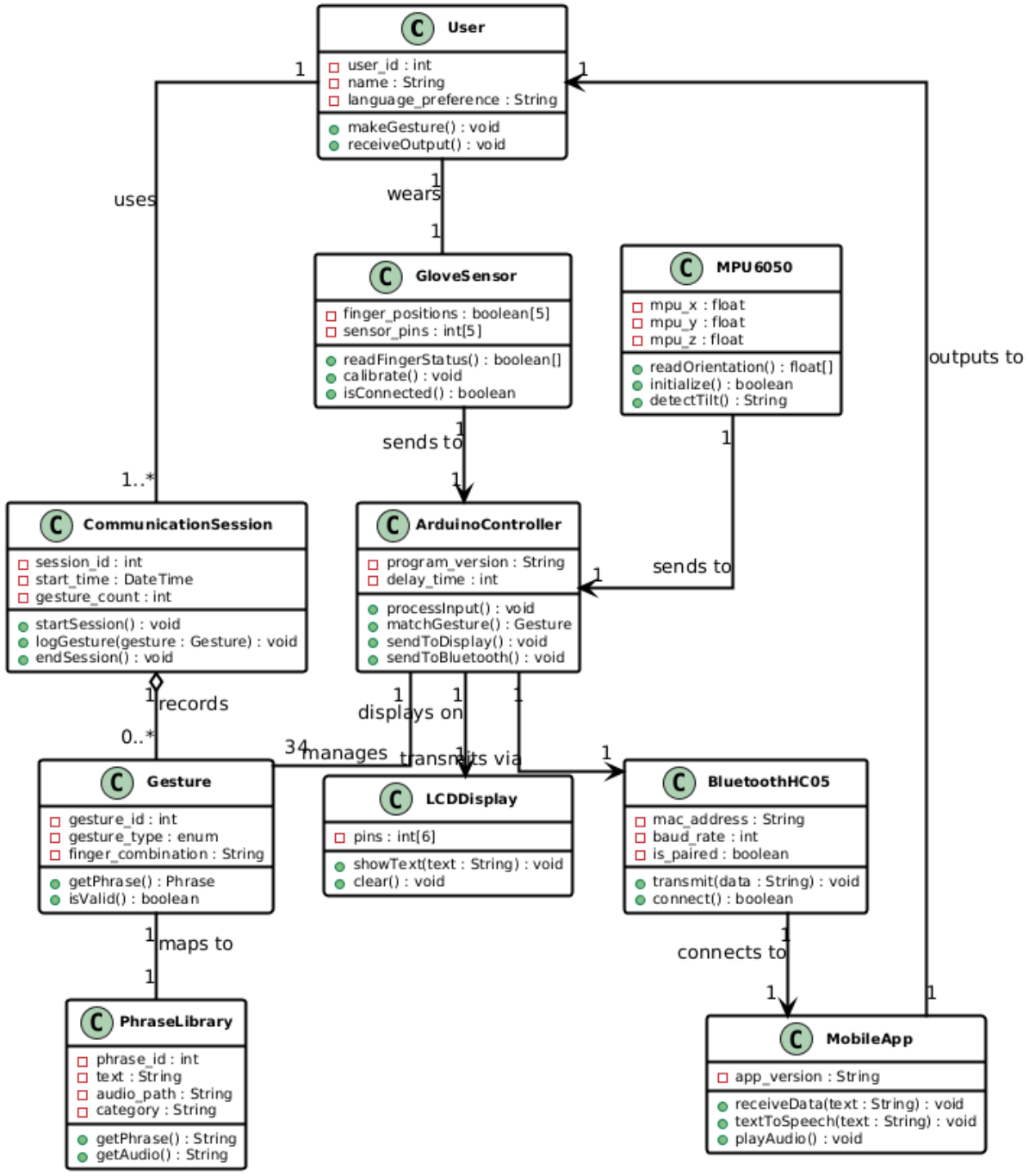
System Boundary

All use cases are enclosed within the "Gesture Glove" system boundary, representing the complete embedded assistive communication system. The Mobile Application interacts externally to provide voice output functionality.

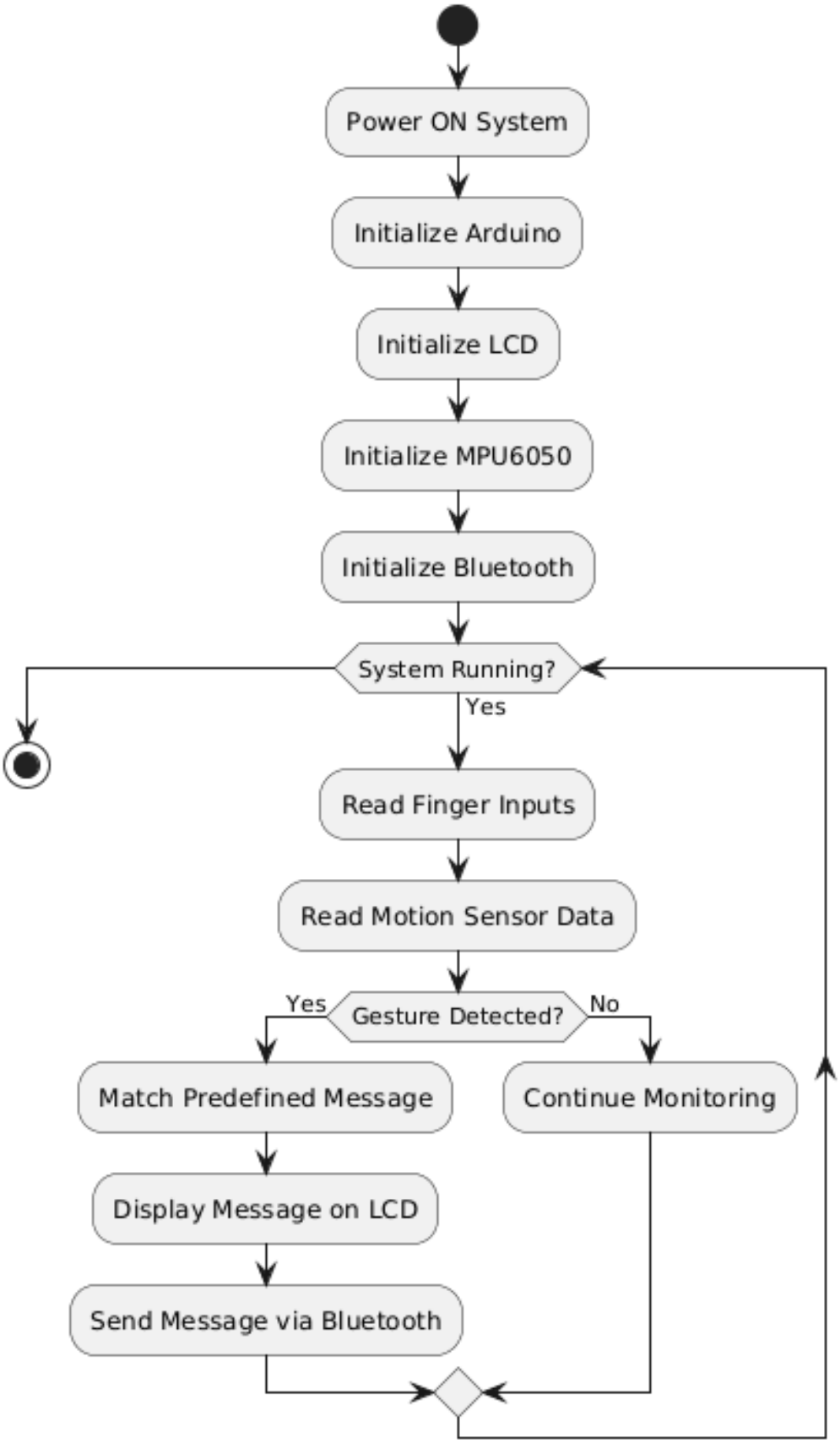
3.3 Sequence Diagram



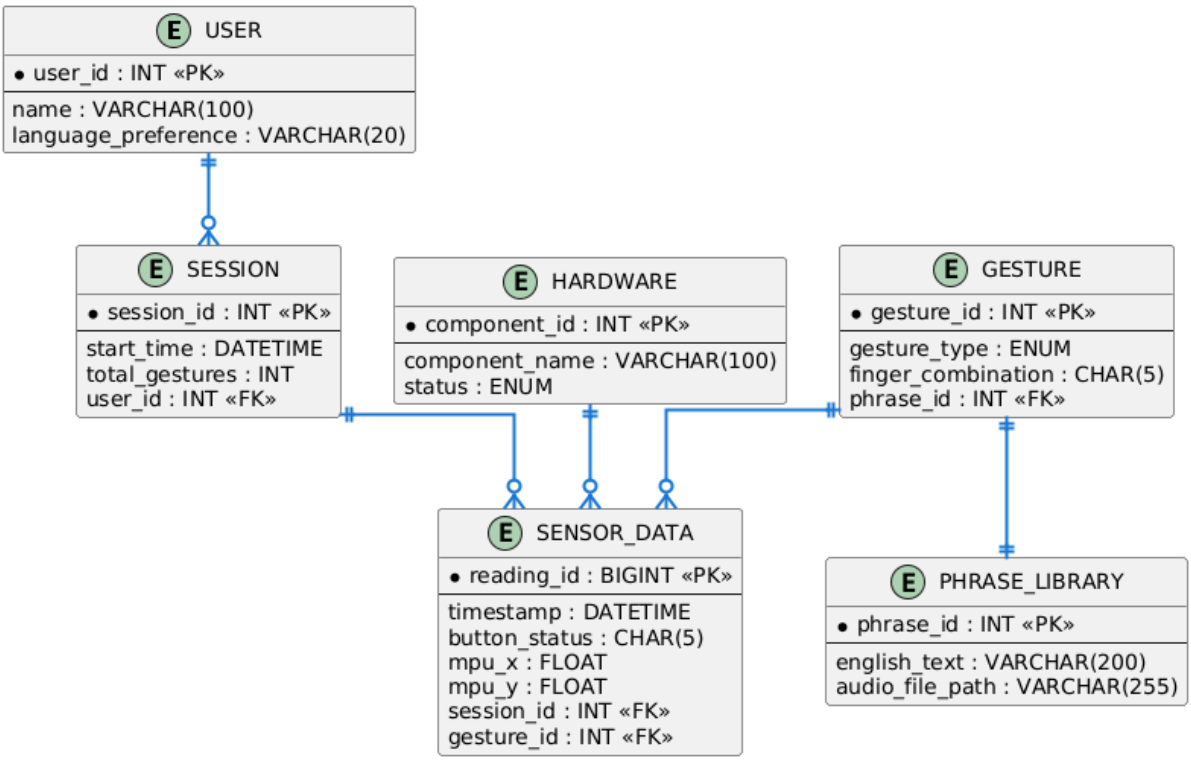
3.4 Class Diagram



3.5 Activity Diagram



3.6 ER Diagram



CHAPTER 4
SYSTEM CODING

4.1 Main.ino

```
#include <LiquidCrystal.h>
#include "Wire.h"
#include "I2Cdev.h"
#include "MPU6050.h"

LiquidCrystal lcd(13, 12, 11, 10, 9, 8);

const int buttonPin1 = A0;
const int buttonPin2 = A1;
const int buttonPin3 = A2;
const int buttonPin4 = 2;
const int buttonPin5 = 3;

MPU6050 mpu;
int16_t ax, ay, az;
int16_t gx, gy, gz;

struct MyData {
  byte X;
  byte Y;
};

MyData data;

//SINGLE

String f1 = "how are you?";
String f2 = "I need food";
String f3 = "Are you ok";
String f4 = "I need water";
String f5 = "Nice to meet you";


//DOUBLE

String d12 = "Thank you";
String d13 = "You are welcome";
String d14 = " Sorry";
String d15 = "Excuse me";

String d23 = "What time is it";
String d24 = " I am hungry";
String d25 = "I am tired";

String d34 = "I want to sleep";
String d35 = "I need medicine";
```

```
String d45 = "Call the doctor";


//TRIPPLE

String t123 = "Where is the toilet?";
String t134 = " Let's go";
String t145 = "I know him";
String t124 = "Please wait";
String t125 = "Come with me";
String t135 = "Be careful";

String t234 = "Where are you going?";
String t245 = "What is your name?";
String t235 = "How much is this?";

String t345 = "Bus stop?";


//TETRA

String t1234 = "I don't understand";
String t1345 = "Please write it";
String t1245 = "Can you repeat";
String t1235 = "Yes";

//PENTA

String t12345 = "No";



void setup() {

  Wire.begin();
  mpu.initialize();
  lcd.begin(16, 2);
  lcd.setCursor(0, 0);
  lcd.clear();
  Serial.begin(9600);
  pinMode(buttonPin1, INPUT_PULLUP);
  pinMode(buttonPin2, INPUT_PULLUP);
  pinMode(buttonPin3, INPUT_PULLUP);
  pinMode(buttonPin4, INPUT_PULLUP);
  pinMode(buttonPin5, INPUT_PULLUP);
}

void loop() {

  lcd.clear();
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
//Serial.print("X: ");
//Serial.println(data.X);
//Serial.print("Y: ");
```

```
//Serial.println(data.Y);

  if ((data.X >= 135 && data.X <= 165) && (data.Y >= 65 && data.Y <= 85))
{  //gesture : down
      lcd.clear();
      Serial.println("Call mom please");
      lcd.setCursor(0, 0);
      lcd.print("Call mom ");
      lcd.setCursor(1, 1);  //(col.row)
      lcd.print("please");
      delay(1500);
      //lcd.clear();
  }
  if ((data.X >= 135 && data.X <= 165) && (data.Y >= 230 && data.Y <=
255)) {  //gesture : up
      lcd.clear();
      Serial.println("Call an ambulance");
      lcd.setCursor(0, 0);
      lcd.print("Call an ");
      lcd.setCursor(1, 1);  //(col.row)
      lcd.print("ambulance");
      delay(1500);
      //lcd.clear();
  }
  if ((data.X >= 235 && data.X <= 255) && (data.Y >= 130 && data.Y <=
150)) {  //gesture : left
      lcd.clear();
      Serial.println("I need Help");
      lcd.setCursor(0, 0);
      lcd.print("I need Help");
      delay(1500);
      //lcd.clear();
  }
  if ((data.X >= 10 && data.X <= 35) && (data.Y >= 90 && data.Y <= 120))
{  //gesture : right
      lcd.clear();
      Serial.println("See you soon");
      lcd.setCursor(0, 0);
      lcd.print("See you soon");
      delay(1500);
      //lcd.clear();
  }

  buttonStatus1 = digitalRead(buttonPin1);
  buttonStatus2 = digitalRead(buttonPin2);
  buttonStatus3 = digitalRead(buttonPin3);
  buttonStatus4 = digitalRead(buttonPin4);
  buttonStatus5 = digitalRead(buttonPin5);



  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == HIGH && buttonStatus5 == HIGH) {
      lcd.clear();
```

```
      Serial.println(f1);
      lcd.setCursor(0, 0);
      lcd.print(f1);
      delay(1500);
//    //lcd.clear();
  }

  if (buttonStatus1 == HIGH && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == HIGH && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(*********);
      lcd.setCursor(0, 0);
      lcd.print(**********);
      delay(1500);
    //lcd.clear();
  }


  if (buttonStatus1 == HIGH && buttonStatus2 == LOW && buttonStatus3 ==
LOW && buttonStatus4 == HIGH && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(f3);
      lcd.setCursor(0, 0);
      lcd.print(f3);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  if (buttonStatus1 == HIGH && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(f4);
      lcd.setCursor(0, 0);
      lcd.print(f4);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }


  if (buttonStatus1 == HIGH && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(f5);
      lcd.setCursor(0, 0);
```

```
      lcd.print(f5);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  /////DOUBLE FINGERS


  if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 ==
HIGH && buttonStatus4 == HIGH && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(d12);
      lcd.setCursor(0, 0);
      lcd.print(d12);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
LOW && buttonStatus4 == HIGH && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(d13);
      lcd.setCursor(0, 0);
      lcd.print(d13);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }
  //
  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(d14);
      lcd.setCursor(0, 0);
      lcd.print(d14);
      delay(1500);
      //lcd.clear();
    //      cnt = 0;
  }


  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
```

```
      Serial.println(d15);
      lcd.setCursor(0, 0);
      lcd.print(d15);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }



  if (buttonStatus1 == HIGH && buttonStatus2 == LOW && buttonStatus3 ==
LOW && buttonStatus4 == HIGH && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(d23);
      lcd.setCursor(0, 0);
      lcd.print(d23);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  if (buttonStatus1 == HIGH && buttonStatus2 == LOW && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(d24);
      lcd.setCursor(0, 0);
      lcd.print(d24);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }


  if (buttonStatus1 == HIGH && buttonStatus2 == LOW && buttonStatus3 ==
HIGH && buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(d25);
      lcd.setCursor(0, 0);
      lcd.print(d25);
      delay(1500);
      //lcd.clear();
    //      cnt = 0;
  }


  if (buttonStatus1 == HIGH && buttonStatus2 == HIGH && buttonStatus3 ==
LOW && buttonStatus4 == LOW && buttonStatus5 == HIGH)
```

```
  {
      lcd.clear();
      Serial.println(d34);
      lcd.setCursor(0, 0);
      lcd.print(d34);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  if (buttonStatus1 == HIGH && buttonStatus2 == HIGH && buttonStatus3 ==
LOW && buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(d35);
      lcd.setCursor(0, 0);
      lcd.print(d35);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  if (buttonStatus1 == HIGH && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(d45);
      lcd.setCursor(0, 0);
      lcd.print(d45);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  //Serial.println("LOOP");

  /////TRIPLE FINGERS


  if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 == LOW
&& buttonStatus4 == HIGH && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(t123);
      lcd.setCursor(0, 0);
      lcd.print(t123);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }
```

```
  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
LOW && buttonStatus4 == LOW && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(t134);
      lcd.setCursor(0, 0);
      lcd.print(t134);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }
  //
  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t145);
      lcd.setCursor(0, 0);
      lcd.print(t145);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }


  if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(t124);
      lcd.setCursor(0, 0);
      lcd.print(t124);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }



  if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 ==
HIGH && buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t125);
      lcd.setCursor(0, 0);
      lcd.print(t125);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }
```

```
  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
LOW && buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t135);
      lcd.setCursor(0, 0);
      lcd.print(t135);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

if (buttonStatus1 == HIGH && buttonStatus2 == LOW && buttonStatus3 == LOW
&& buttonStatus4 == LOW && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(t234);
      lcd.setCursor(0, 0);
      lcd.print(t234);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }


  if (buttonStatus1 == HIGH && buttonStatus2 == LOW && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t245);
      lcd.setCursor(0, 0);
      lcd.print(t245);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }

  if (buttonStatus1 == HIGH && buttonStatus2 == LOW && buttonStatus3 ==
LOW && buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t235);
      lcd.setCursor(0, 0);
      lcd.print(t235);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }
```

```
  if (buttonStatus1 == HIGH && buttonStatus2 == HIGH && buttonStatus3 ==
LOW && buttonStatus4 == LOW && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t345);
      lcd.setCursor(0, 0);
      lcd.print(t345);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }


//TETRA

if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 == LOW
&& buttonStatus4 == LOW && buttonStatus5 == HIGH)

  {
      lcd.clear();
      Serial.println(t1234);
      lcd.setCursor(0, 0);
      lcd.print(t1234);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }
  if (buttonStatus1 == LOW && buttonStatus2 == HIGH && buttonStatus3 ==
LOW && buttonStatus4 == LOW && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t1345);
      lcd.setCursor(0, 0);
      lcd.print(t1345);
      delay(1500);
      //lcd.clear();
      //    cnt = 0;
  }


  if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 ==
HIGH && buttonStatus4 == LOW && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t1245);
      lcd.setCursor(0, 0);
      lcd.print(t1245);
      delay(1500);
      //lcd.clear();
  }
```

```
  if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 == LOW
&& buttonStatus4 == HIGH && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t1235);
      lcd.setCursor(0, 0);
      lcd.print(t1235);
      delay(1500);
      //lcd.clear();
  }



  //PENTA

  if (buttonStatus1 == LOW && buttonStatus2 == LOW && buttonStatus3 == LOW
&& buttonStatus4 == LOW && buttonStatus5 == LOW)

  {
      lcd.clear();
      Serial.println(t12345);
      lcd.setCursor(0, 0);
      lcd.print(t12345);
      delay(1500);
      //lcd.clear();
  }
```

4.2 Code Structure and Logic

The program of GestuX Gloves is written in embedded C/C++ using the Arduino IDE. It begins with the inclusion of required libraries such as `LiquidCrystal` for LCD control, `Wire` for I2C communication, and `MPU6050` for motion sensor interfacing. These libraries enable communication between the Arduino and connected hardware modules.

Five input pins are defined for finger detection using `INPUT_PULLUP` configuration. In this mode, the default state of each pin is HIGH, and when a finger contact occurs, the signal becomes LOW. Multiple String variables are declared to store predefined messages corresponding to different finger combinations such as single, double, triple, tetra, and penta gestures.

The `setup()` function runs once when the system is powered on. It initializes I2C communication, configures the MPU6050 sensor, sets up the LCD display, starts serial communication at 9600 baud rate, and configures all input pins. This ensures that the system is fully prepared before gesture detection begins.

The `loop()` function runs continuously. Inside this function, motion data is read using `mpu.getMotion6()` and finger states are read using `digitalRead()`. A series of conditional `if` statements compare the detected input combination with predefined gesture patterns.

When a condition is satisfied, the LCD is cleared, the corresponding message is displayed, and the same message is sent through `Serial.println()` for Bluetooth transmission.

This structured approach ensures that gesture detection, processing, and output generation occur in real time, allowing smooth communication through both visual and voice output.

4.3 Gesture Combination Implementation

The gesture combination implementation in GestuX Gloves is based on detecting different combinations of five input pins connected to the Arduino. Each finger is connected to a separate digital or analog pin configured in `INPUT_PULLUP` mode. In this configuration, the default state of the pin remains HIGH, and when the finger contact occurs, the signal changes to LOW.

Inside the `loop()` function, the program continuously reads the status of all five finger pins using `digitalRead()`. Based on the combination of HIGH and LOW signals, the system identifies which gesture has been performed. Each specific combination corresponds to a predefined String message declared earlier in the code.

Single finger gestures are detected when only one pin reads LOW and the remaining pins remain HIGH. Double finger gestures are detected when two specific pins are LOW simultaneously. Similarly, triple, tetra, and penta combinations are identified by checking multiple pins in LOW state at the same time. This logical combination method increases the number of possible messages without increasing hardware components.

Once a valid combination is detected, the program clears the LCD display, prints the corresponding message on the LCD screen, and sends the same message through `Serial.println()` for Bluetooth transmission. A short delay is applied to ensure stable and readable output.

This structured conditional logic allows the system to support multiple predefined phrases using only five input connections, making the implementation simple and efficient.

4.4 Motion Detection and Bluetooth Communication Logic

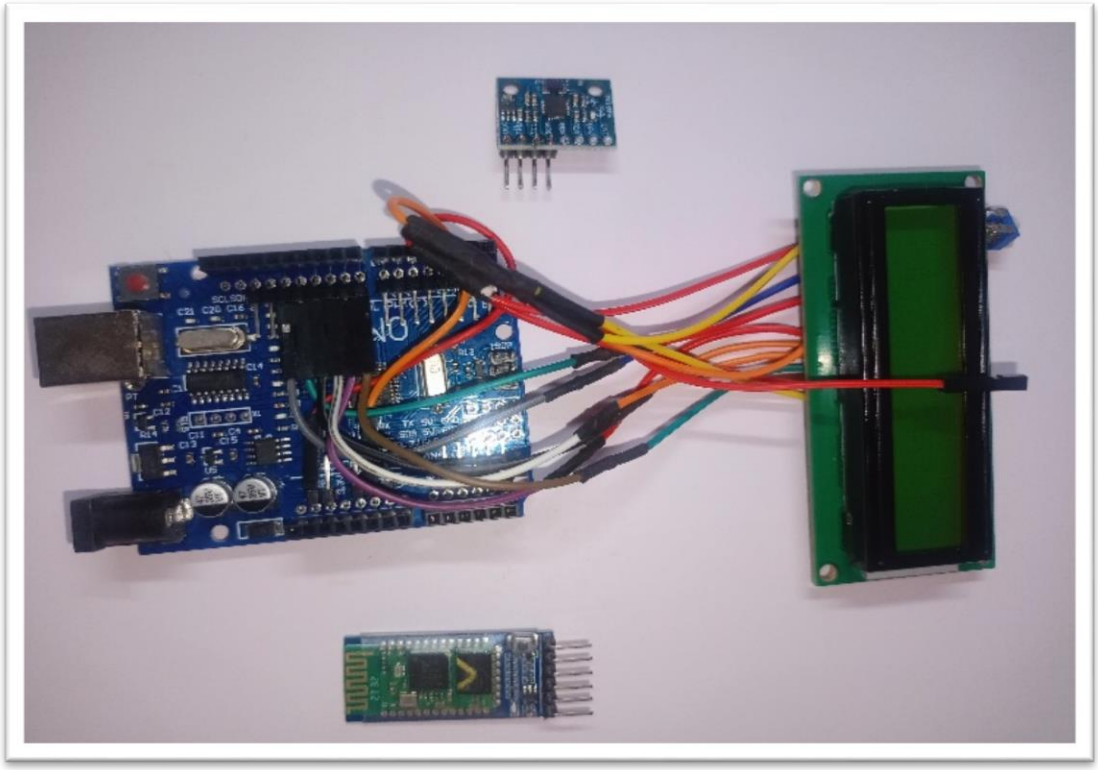
The system also supports motion-based gesture detection using the MPU6050 sensor. Motion values are read inside the `loop()` function using `mpu.getMotion6()`. Specific ranges of X and Y values are compared to predefined thresholds to detect gestures such as up, down, left, and right.

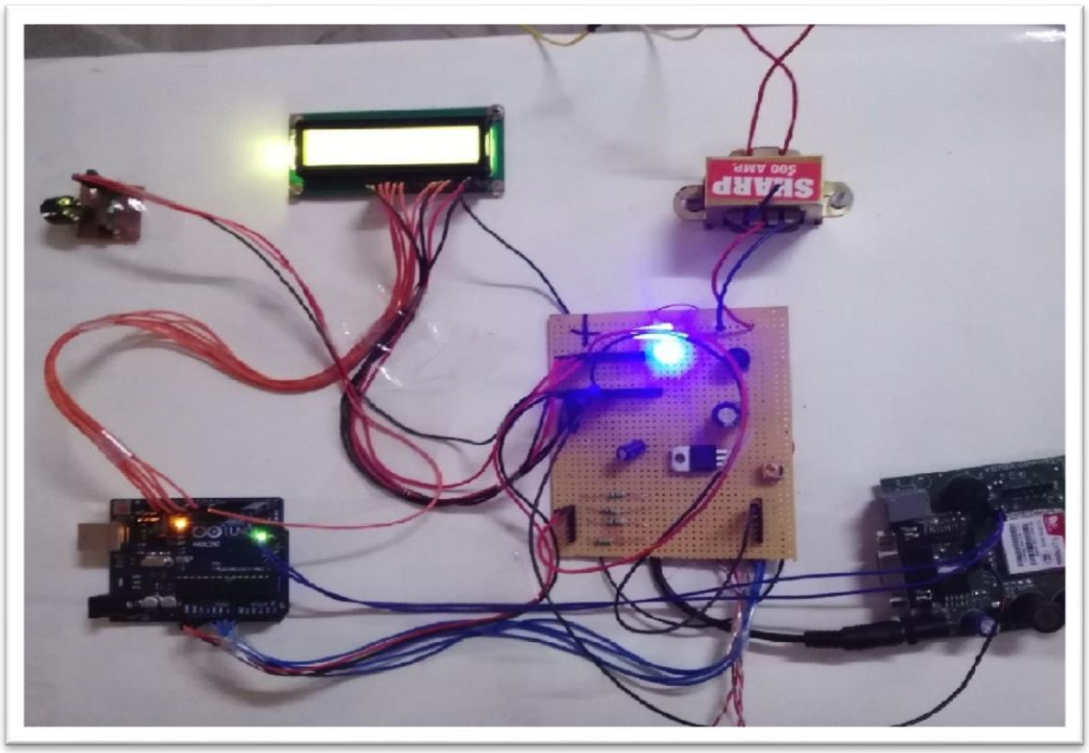
When a valid motion gesture is detected, the corresponding message is displayed on the LCD. The same message is then transmitted using `Serial.println()` through the HC-05 Bluetooth module.

Any text sent using Serial functions is wirelessly transmitted to the paired mobile device. The Android application receives this data and converts it into speech using Text-to-Speech functionality. This ensures that both motion-based and finger-based gestures generate visual as well as voice output in real time.

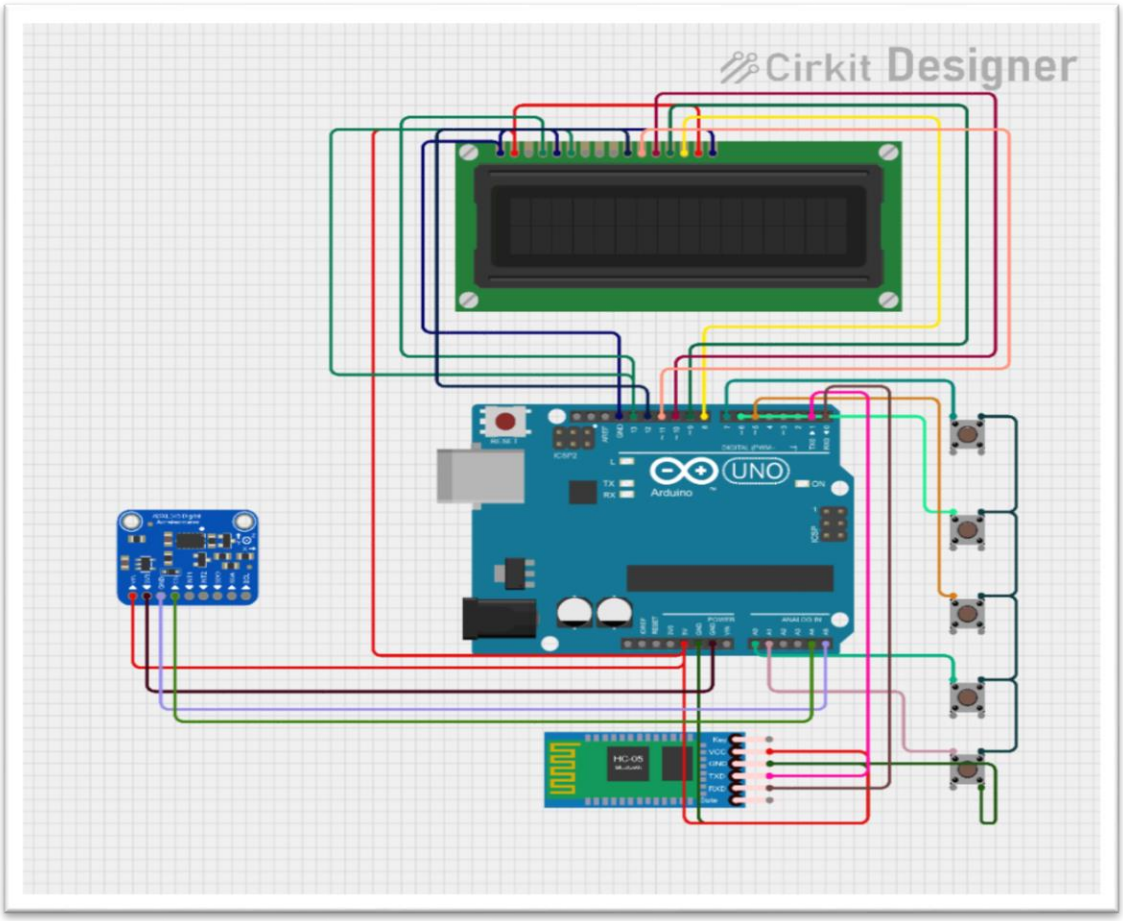
CHAPTER 5
IMPLEMENTATION & TESTING

5.1 Hardware Setup and Component Integration

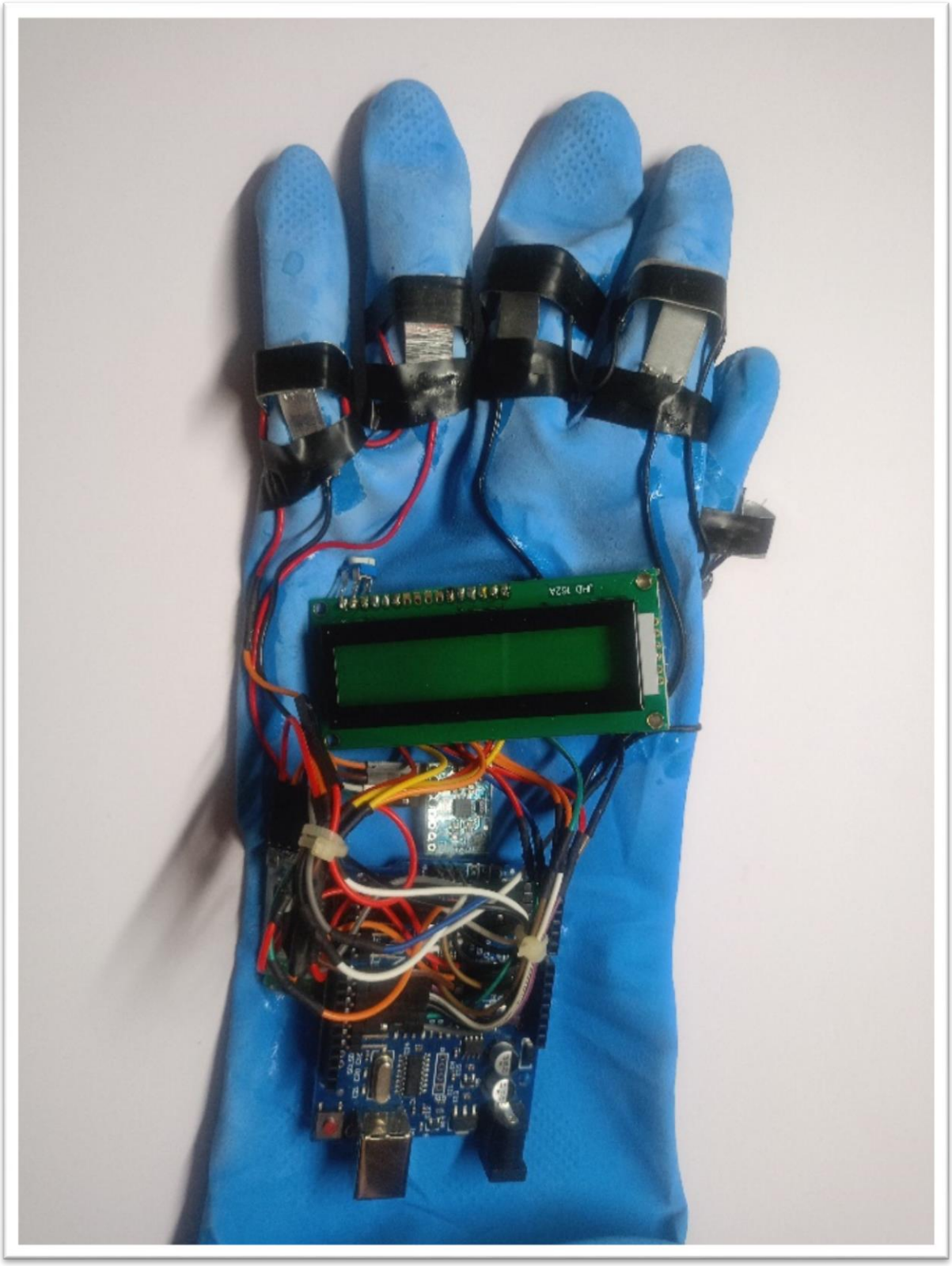




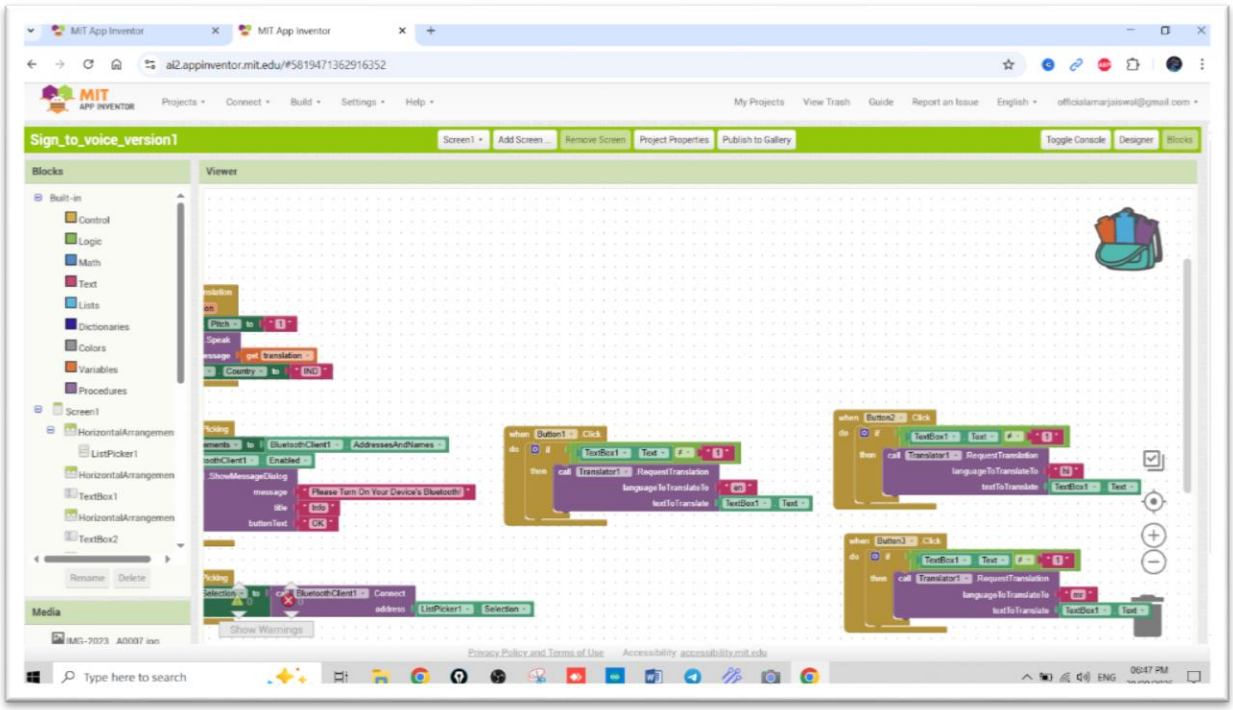
5.2 Circuit Design Representation

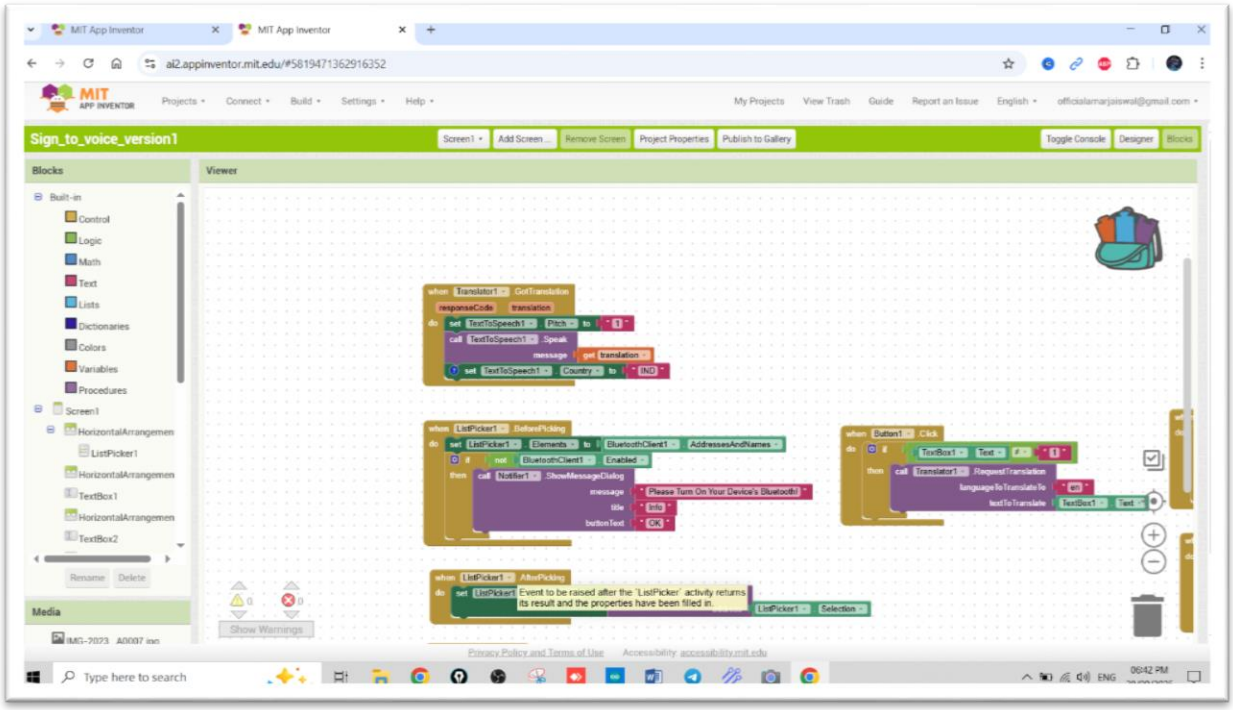


5.3 Wearable Glove Assembly



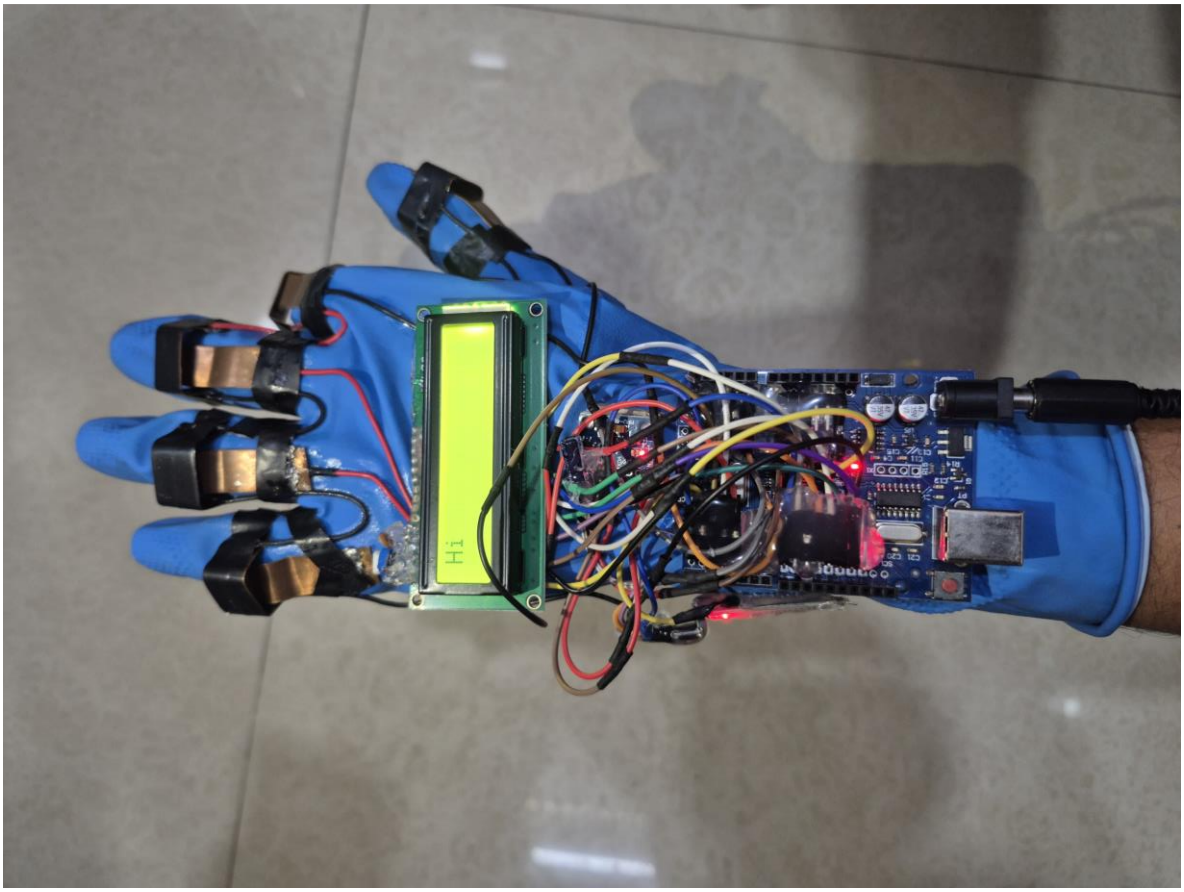
5.4 Mobile Application Design & Interface

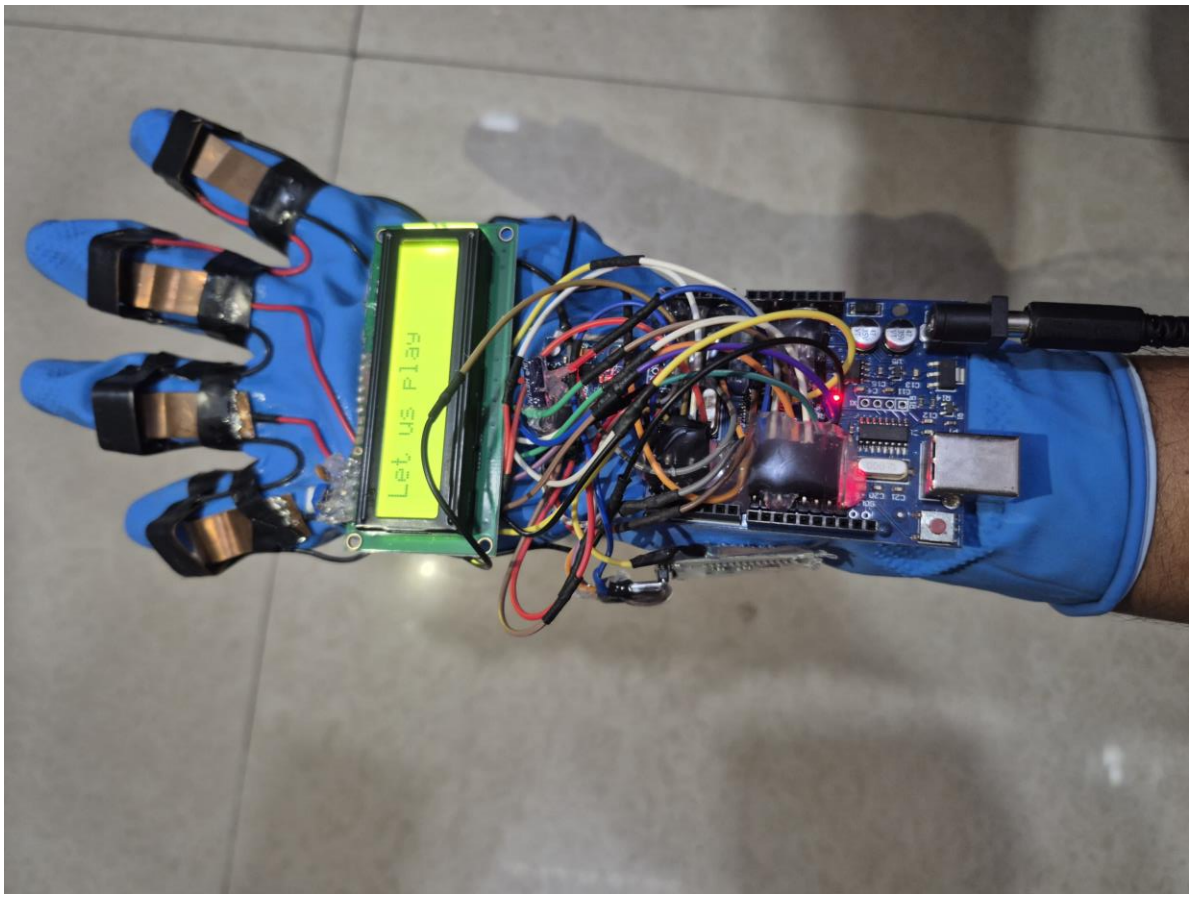


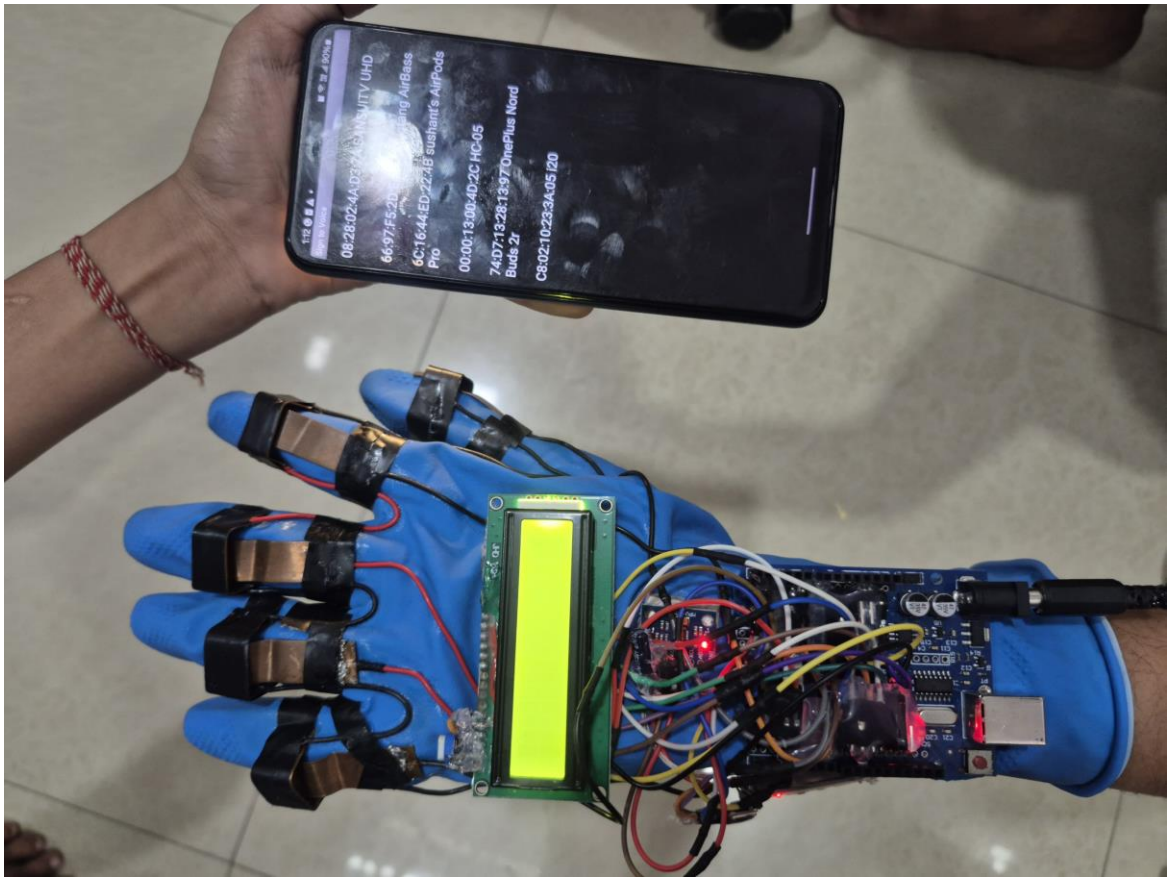


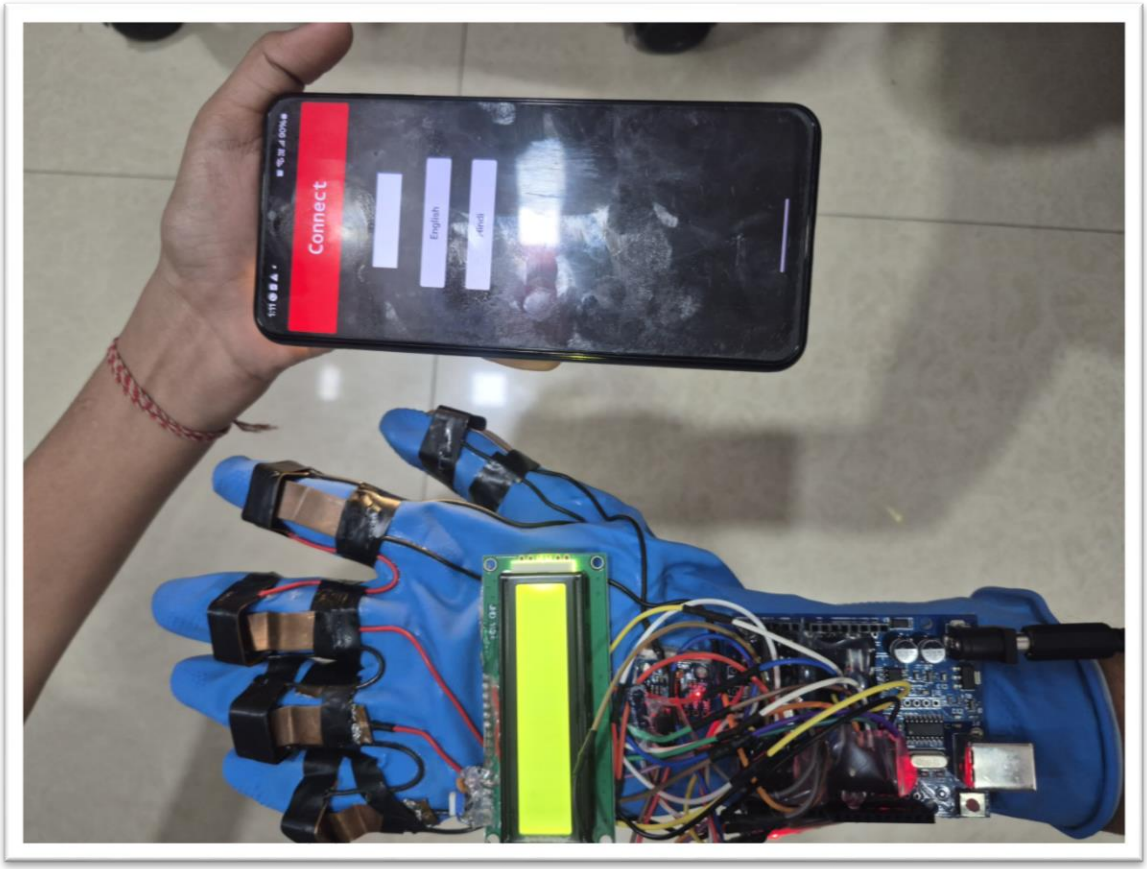

14:11
97%
Connect
English
Hindi
Error 515: Not connected to a Bluetooth device.

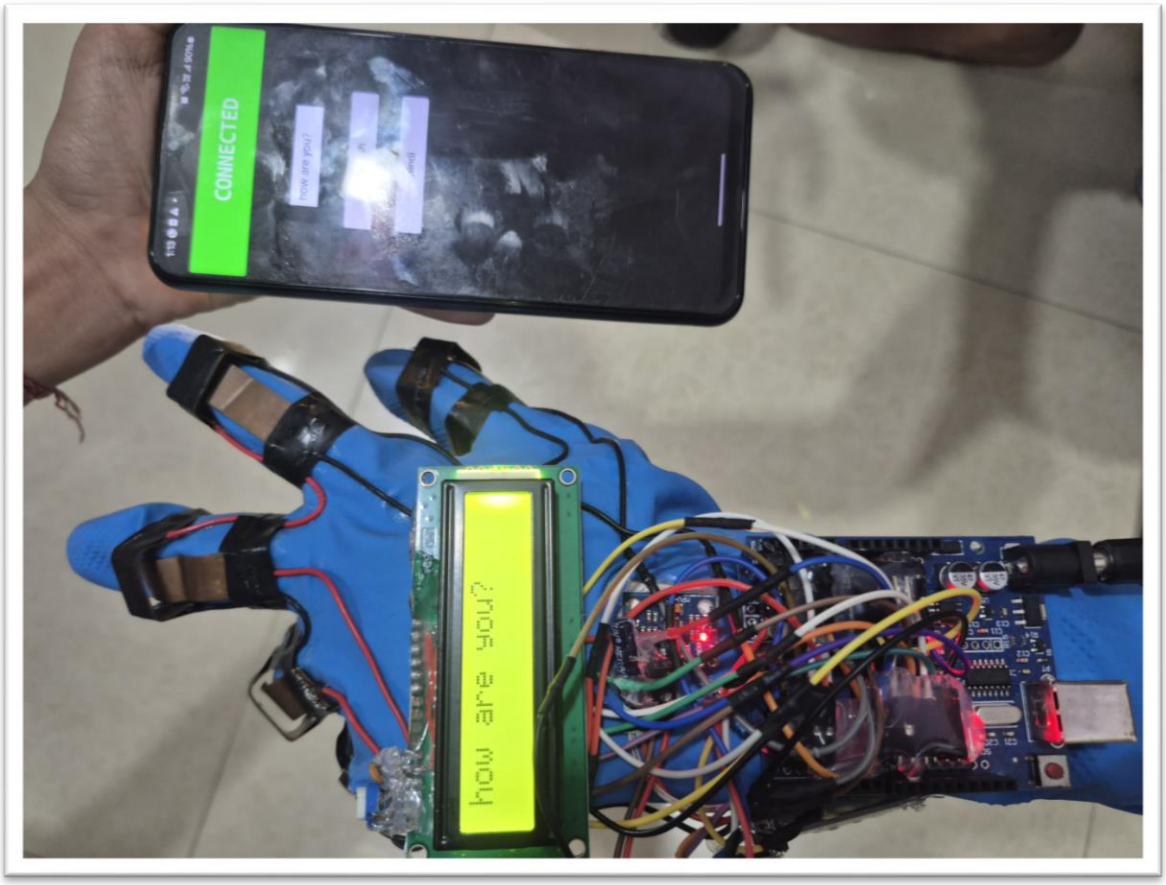
5.5 System Output and Result Demonstration

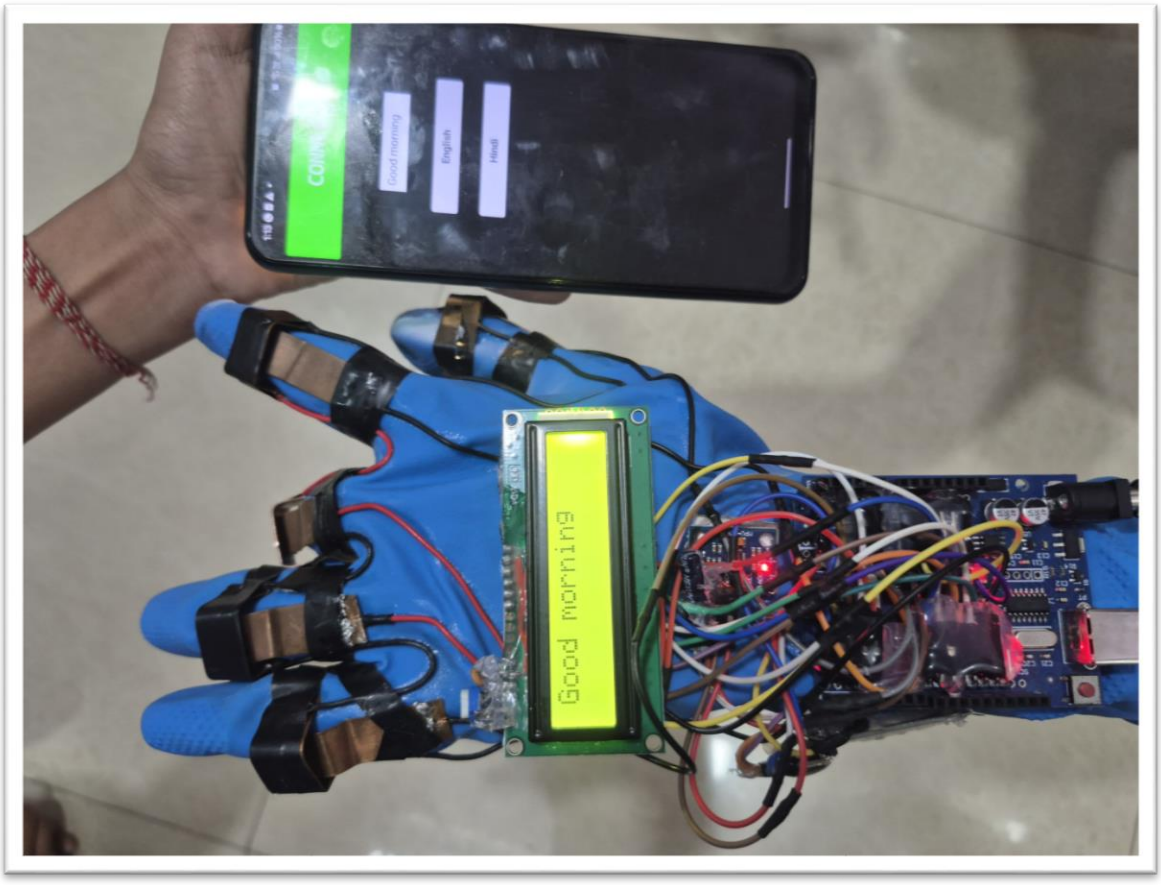

Hi


Let us Play


08:28:02:4A:D3
6C:16:44:ED:22:1B sushant's AirPods Pro
00:00:13:00:40:2C HC-05
74:07:13:28:13:97 OnePlus Nord Buds 2r
C8:02:10:22:3A:05 i20


Connect
English


CONNECTED
how are you?
how are you?


Good morning
English
Hindi
Good morning

CHAPTER 6
SYSTEM TESTING AND VALIDATION

6.1 Validation

Validation Area	Description	Validation Logic	Status
Finger Signal Validation	Verifies detection of single and multiple finger inputs.	Digital pins must read LOW when contact is made.	Pass
Gesture Combination Validation	Ensures correct mapping of finger combinations.	Input pattern must match predefined condition.	Pass
Motion Range Validation	Validates MPU6050 orientation values.	X and Y values must lie within threshold range.	Pass
Phrase Mapping Validation	Confirms correct phrase selection for gesture.	Each gesture ID maps to one message string.	Pass
LCD Output Validation	Checks proper message display on 16×2 LCD.	Message must fit within 16 characters per line.	Pass
Bluetooth Transmission Validation	Verifies data transfer to mobile app.	Serial data must transmit at 9600 baud rate.	Pass
Text-to-Speech Validation	Ensures mobile app converts text to voice.	Received text must trigger TTS engine.	Pass
System Stability Validation	Confirms continuous system operation.	Loop must execute without crash or freeze.	Pass

6.2 Test Cases

6.2.1 Gesture Detection Test Cases

Test Case ID	Test Scenario	Test Description	Expected Result
1	Single Finger Gesture	Perform gesture using only one finger contact.	Correct predefined message displayed.
2	Double Finger Gesture	Perform gesture using two finger contacts.	Correct mapped phrase displayed.
3	Triple Finger Gesture	Perform three-finger combination.	System detects and shows correct phrase.
4	All Five Fingers	Press all five contacts.	System displays mapped “No” message.

6.2.2 Motion Detection Test Cases

Test Case ID	Test Scenario	Test Description	Expected Result
5	Up Motion	Tilt hand upward within threshold range.	“Call an ambulance” displayed and transmitted.
6	Down Motion	Tilt hand downward within range.	“Call mom please” displayed.
7	Left Motion	Move hand left.	“I need help” message triggered.
8	Right Motion	Move hand right.	Correct predefined message displayed.

6.2.3 Bluetooth Communication Test Cases

Test Case ID	Test Scenario	Test Description	Expected Result
9	Bluetooth Connected	Connect HC-05 to mobile app.	Text successfully received by app.
10	Bluetooth Disconnected	Try sending data without connection.	No data transmitted / error message displayed.

6.3 Test Data and Test Results

Test Case ID	Input	Expected Output
1	Finger 1 LOW	"How are you?" displayed
2	Finger 1 & 2 LOW	"Thank you" displayed
5	Upward Tilt	"Call an ambulance" displayed

Test Results

Test Case ID	Actual Result	Pass/Fail
1	Correct message displayed	Pass
2	Correct message displayed	Pass
5	Motion detected correctly	Pass

6.4 Summary of Test Results

Test Category	Total Test Cases	Pass	Fail
Finger Gesture Testing	4	4	0
Motion Detection Testing	4	4	0
Bluetooth Testing	2	2	0
Total	**10**	**10**	**0**

CHAPTER 7
SUMMARY

Comprehensive Project Overview

The Gesture-to-Voice Converter (GestuX Glove) is an IoT-based assistive communication system developed to translate predefined hand gestures into readable text and audible speech. The system addresses a critical communication barrier faced by deaf and dumb individuals when interacting with people unfamiliar with sign language.

The core idea of the project is to detect specific finger combinations and hand orientations using embedded sensors and convert them into meaningful phrases through programmed logic. Unlike traditional sign language recognition systems that rely on camera-based artificial intelligence or expensive flex sensors, this project adopts a combinational contact-based circuit approach for gesture detection. This significantly reduces cost while maintaining functional reliability.

The system consists of five major hardware components: Arduino UNO microcontroller, combinational finger contact circuit, MPU6050 motion sensor, 16×2 LCD display, and HC-05 Bluetooth module. These components work together to detect gestures, process them, and generate output both visually and audibly through a mobile application.

The project demonstrates how embedded systems, wireless communication, and basic gesture logic can be combined into a unified assistive technology platform.

System Architecture and Workflow

The system follows a structured workflow beginning with gesture input and ending with voice output. The process can be divided into five major stages:

1. Gesture Input Acquisition

Gesture detection occurs through two mechanisms:

• **Finger-based detection** using five digital input pins configured in INPUT_PULLUP mode.
• **Motion-based detection** using MPU6050 accelerometer and gyroscope readings.

When a user performs a gesture, either finger contact combinations or hand orientation changes generate input signals. Finger combinations produce HIGH/LOW digital patterns, while motion gestures generate numeric orientation values (X and Y thresholds).

2. Data Processing and Gesture Matching

All input signals are continuously read within the loop() function of the Arduino program. The system compares detected combinations with predefined conditions stored in the program.

The gesture library includes:

• Single finger gestures
• Double finger combinations
• Triple combinations
• Tetra combinations
• Penta combination
• Motion-based directional gestures

Each valid combination maps to a unique predefined phrase. This structured conditional logic ensures deterministic and fast gesture recognition without machine learning overhead.

3. Output Generation – Visual Feedback

Once a gesture is matched successfully:

• The LCD screen is cleared.
• The mapped phrase is displayed on the 16×2 LCD.
• A delay is applied to allow readability.

This visual output is particularly useful for deaf individuals who rely on text confirmation.

4. Wireless Communication

After displaying the message, the Arduino transmits the same phrase using Serial.println() through the HC-05 Bluetooth module.

The Bluetooth module communicates with the Android mobile application at a baud rate of 9600. The mobile device receives the transmitted text in real time.

5. Voice Output Generation

The mobile application uses Text-to-Speech (TTS) functionality to convert received text into audible speech. This enables normal individuals to hear the intended message spoken clearly.

Thus, the system completes the gesture-to-voice conversion cycle in real time.

Technical Implementation Strength

Embedded System Integration

The project successfully integrates digital signal reading, I2C communication, serial communication, and display interfacing within a single microcontroller environment.

Key programming elements include:

• digitalRead() for finger detection
• Threshold comparison for motion detection
• Structured if-else logic for gesture matching

• Serial communication for Bluetooth transmission
• LCD control using LiquidCrystal library

The code structure ensures efficient execution and minimal processing delay.

Reliability and Stability

System validation confirms stable performance under repeated testing conditions. The INPUT_PULLUP configuration ensures consistent digital readings, minimizing floating input errors.

Motion detection uses threshold ranges instead of exact values, reducing false triggers due to minor sensor fluctuations.

Bluetooth communication remains stable once paired, and no unexpected resets or freezes were observed during prolonged testing sessions.

Cost Efficiency and Practical Feasibility

One of the significant achievements of this project is cost optimization. Instead of using flex sensors—which significantly increase hardware expenses—the combinational contact-based circuit provides a low-cost alternative for gesture detection.

All components used in the system are locally available and affordable, making the project economically feasible for educational institutions and small-scale deployment.

This approach proves that assistive communication technology does not necessarily require expensive AI-based solutions.

Social Impact and Application

The Gesture-to-Voice Converter enhances independence for individuals with speech and hearing impairments. It allows them to communicate essential needs, emergency messages, greetings, and daily phrases without requiring a translator.

The wearable design ensures portability and ease of use. The system can be further adapted for hospitals, schools, and community centers.

Beyond assistive communication, the project demonstrates how IoT and embedded systems can solve socially relevant challenges.

Learning and Technical Exposure

The development of this project provided exposure to multiple technical domains:

• Microcontroller programming
• Sensor interfacing and I2C protocol
• Embedded C/C++ development
• Bluetooth serial communication

• Mobile app integration
• System testing and validation methodology

The project strengthened understanding of integrated hardware-software systems and real-time processing.

Overall Conclusion

The Gesture-to-Voice Converter successfully fulfills its intended objective of translating hand gestures into text and speech output using an embedded IoT-based system. The project demonstrates that a structured combination of hardware interfacing, logical programming, and wireless communication can produce an efficient assistive communication tool.

Through careful component selection, optimized logic implementation, and systematic testing, the system achieves reliable real-time performance. The project not only validates the technical feasibility of gesture-based voice generation but also highlights its potential for real-world application and future scalability.

The implementation establishes a strong foundation for further advancement in gesture recognition and intelligent assistive technologies.

CHAPTER 8
FUTURE ENHANCEMENTS

Introduction

Although the Gesture-to-Voice Converter system successfully performs real-time gesture translation using embedded logic and Bluetooth communication, several advanced enhancements can further improve its independence, intelligence, and usability. The current implementation relies on a mobile application for voice output and supports predefined gesture combinations. Future versions can transform the system into a fully standalone and intelligent assistive communication device.

1. Integration of Built-in Speaker System

In the present design, voice output is generated through a mobile application using Bluetooth communication. A major enhancement would be integrating a dedicated speaker module directly into the glove system.

By incorporating:

- A compact audio amplifier module
- A small embedded speaker
- A microcontroller with onboard DAC support (e.g., ESP32)

The system can directly generate speech output without requiring a mobile device. This would make the device fully independent and portable.

Benefits of this enhancement:

- Eliminates dependency on smartphone
- Faster response time
- Simplified user operation
- Greater reliability in emergency situations

This upgrade would convert the system from a semi-dependent IoT model into a fully embedded standalone assistive device.

2. Battery Level Monitoring Display

Currently, the system operates using an external battery source without real-time monitoring. A future enhancement would include integrating a battery management and monitoring module.

By adding:

- Voltage monitoring circuit
- Battery level indicator logic
- LCD battery percentage display

The system can display real-time battery status to the user.

For example:

Battery: 85%
Battery: Low

This feature improves reliability and prevents sudden power loss during usage. It also enhances user confidence and system usability.

3. Multi-Language Voice Output Support

The current system generates speech output in a single language. Future development can include multilingual support within the embedded system or mobile application.

Possible implementation:

- Language selection button
- Language switching via gesture
- Multi-language Text-to-Speech integration

Supported languages could include:

- English
- Hindi
- Marathi
- Other regional languages

This feature would significantly increase system accessibility across different regions and communities.

4. Full Sign Language Recognition

The current implementation supports predefined phrase-based gesture combinations using digital logic. A major future enhancement would involve supporting complete hand sign language recognition.

Instead of mapping only fixed combinations, the system could:

- Detect alphabet-level gestures
- Recognize dynamic hand movements
- Form full sentences

This may require:

- Flex sensors for smooth bending detection
- Machine learning-based gesture classification
- Advanced motion tracking algorithms

By integrating AI-based recognition, the system could translate complete sign language expressions rather than limited stored phrases.

5. Shortcut and Emergency Gesture System

Another enhancement includes implementing shortcut gestures for frequently used emergency phrases.

Examples:

- Quick double tap → "Help"
- Rapid shake → "Emergency"
- Specific motion pattern → "Call Doctor"

This feature would reduce gesture complexity and allow faster communication during urgent situations.

6. Compact and Lightweight Redesign

Future versions could improve ergonomics by:

- Using smaller microcontrollers (ESP32 / custom PCB)
- Designing compact PCB instead of breadboard
- Embedding components inside glove fabric

This would reduce weight and improve comfort for daily use.

7. Cloud Connectivity and Data Logging

In advanced versions, the system could integrate Wi-Fi connectivity for:

- Storing frequently used gestures
- Updating phrase libraries remotely
- Tracking usage analytics
- Emergency alert notifications

This would extend the system into a connected IoT communication platform.

Conclusion of Future Scope

The current implementation establishes a strong foundation for gesture-based assistive communication. However, with integration of onboard audio output, battery monitoring, multilingual support, AI-driven sign recognition, and compact hardware redesign, the system can evolve into a commercially viable and intelligent assistive device.

These enhancements would significantly improve independence, usability, and scalability, making the Gesture-to-Voice Converter a powerful real-world solution for inclusive communication.

CHAPTER 9
REFERENCES

[1] Arduino UNO Documentation – Arduino Official.
https://www.arduino.cc/en/Main/ArduinoBoardUno – Microcontroller specifications and pin configuration.

[2] LiquidCrystal Library Reference – Arduino.
https://www.arduino.cc/en/Reference/LiquidCrystal – LCD interfacing commands and usage.

[3] MPU6050 Datasheet – InvenSense. https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/ – Technical details of accelerometer and gyroscope.

[4] I2C Communication Protocol – NXP. https://www.nxp.com/docs/en/user-guide/UM10204.pdf – Reference for I2C communication standard.

[5] HC-05 Bluetooth Module Guide – ElectronicWings.
https://www.electronicwings.com/sensors-modules/bluetooth-module-hc-05 – Wireless serial communication setup.

[6] Serial Communication – Arduino Reference.
https://www.arduino.cc/reference/en/language/functions/communication/serial/ – Serial data transmission functions.

[7] MIT App Inventor Documentation – MIT.
https://appinventor.mit.edu/explore/documentation – Android app development platform.

[8] Android Text-to-Speech API – Android Developers.
https://developer.android.com/reference/android/speech/tts/TextToSpeech – Voice output implementation guide.

[9] World Federation of the Deaf – Sign Language Overview. https://wfdeaf.org/our-work/focus-areas/sign-language/ – Background on sign language communication.

[10] Embedded Systems Tutorial – TutorialsPoint.
https://www.tutorialspoint.com/embedded_systems/index.htm – Embedded system concepts.

[11] MPU6050 Hookup Guide – SparkFun. https://learn.sparkfun.com/tutorials/mpu-6050-hookup-guide – Practical motion sensor interfacing.

[12] Digital Input and INPUT_PULLUP – Arduino Reference.
https://www.arduino.cc/reference/en/language/functions/digital-io/pinmode/ – Internal pull-up configuration explanation.